In the name of Allah

Data Networks

Prof. Mohammad Reza Pakravan

Assignment 4

Farzan Aghaei 401101202

Sharif University of Technology

Electrical Engineering Department

1 Analyzing Protocols Using Wireshark

1.1 Analyze IP Protocol

1.1.1 Simulation Scenario: Ping Your Gateway and Analyze IP Packets

I first started capturing traffic on WiFi interface. After that I opened the command prompt and then used the *ipconfig* command to find out that my default gateway IP's address is 192.168.100.1 . Then using *ping 192.168.100.1* I ran a ping command in that gateway and after it completed I stopped the capture.

The results of the process above are shown in the following figure.

```
C:\Users\juve2>ping 192.168.100.1

Pinging 192.168.100.1 with 32 bytes of data:
Reply from 192.168.100.1: bytes=32 time=1ms TTL=64
Reply from 192.168.100.1: bytes=32 time=1ms TTL=64
Reply from 192.168.100.1: bytes=32 time=2ms TTL=64
Reply from 192.168.100.1: bytes=32 time=2ms TTL=64

Ping statistics for 192.168.100.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 2ms, Average = 1ms
```

Figure 1: default gateway ping process result

Next I filtered the ICMP packets, chose the first Echo Request one and then expanded its IP section which can be observed below.

```
▸ Frame 78: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface \Device\NPF_{CE25FC99-6C9B-4150-9535-1E9A9285CE08}, id 0
▸ Ethernet II, Src: ChongqingFug_3a:f1:4b (a8:93:4a:3a:f1:4b), Dst: HuaweiTechno_16:4b:07 (a0:a3:3b:16:4b:07)
▾ Internet Protocol Version 4, Src: 192.168.100.9, Dst: 192.168.100.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▾ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
      0000 00.. = Differentiated Services Codepoint: Default (0)
      .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
    Total Length: 60
    Identification: 0xe9f4 (59892)
  ▾ 000. .... = Flags: 0x0
      0... .... = Reserved bit: Not set
      .0.. .... = Don't fragment: Not set
      ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: ICMP (1)
    Header Checksum: 0x0771 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.100.9
    Destination Address: 192.168.100.1
    [Stream index: 2]
▸ Internet Control Message Protocol
```

Figure 2: The IP section of the first captured ICMP packet

1.1.2 Questions: IP Header Field Analysis

1. **Header Length:** What is the header length in bytes?

   As it can be seen in 2, the header length is 5 bytes.

2. **Type of Service (DSCP/ECN):** What value is set? What is the purpose of this field?

   The value set to this field is 0x00. This field is used for Quality of Service. DSCP specifies packet priority, and ECN is used for congestion notification in networks.

3. **Total Length:** What is the size of the entire IP packet?

   The total length is 60 bytes.

4. **Identification:** What is the purpose of this field?

   This filed is set to 0xe9f4 (59892) .This field is used to uniquely identify fragments of an original IP datagram. It helps the receiving system reassemble packets that have been fragmented during transmission.

5. **Flags and Fragment Offset:** Are any flags set? What do they mean?

   The flags field is set to 0x0 which tells us that there are no flags set. This means that no fragmentation is occurring, this packet is not fragmented and there are no restrictions on fragmentation.

6. **Time to Live (TTL):** What is the TTL value? Why is this important?

   The TTL value is 128. TTL limits the lifetime of a packet to prevent it from circulating indefinitely in case of routing loops. Each router that forwards the packet decreases the TTL by 1; when it reaches 0, the packet is discarded.

7. **Protocol:** Which Layer 4 protocol is encapsulated in this IP packet?

   The protocol is ICMP (1). It is commonly used for diagnostics (e.g., ping).

8. **Source and Destination IPs:** Do these match your IP and gateway?

   The source address is 192.168.100.9 and the destination address is 192.168.100.1 and they match my computer's IP and gateway.

1.2 Analyze DHCP Protocol

1.2.1 Simulation Scenario

Again in this part I started capturing packets on my WiFi interface. Then I released and renewed my IP address to initiate DHCP exchange using the commands *ipconfig /release* and *ipconfig /renew*.

```
Wireless LAN adapter WiFi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::711f:3bc5:35cc:4766%18
   IPv4 Address. . . . . . . . . . . : 192.168.100.9
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.100.1
```

Figure 3: The Wireless LAN adapter WiFi after release and renew

After a few seconds I stopped the packet capturing process and applied the dhcp display filter on the captured packets.

```
199 23.520327    192.168.100.9    192.168.100.1    DHCP    342 DHCP Release  - Transaction ID 0x6f604690
318 32.097914    0.0.0.0          255.255.255.255  DHCP    342 DHCP Discover - Transaction ID 0x3d9a8e71
366 34.181332    192.168.100.1    192.168.100.9    DHCP    590 DHCP Offer    - Transaction ID 0x3d9a8e71
367 34.182227    0.0.0.0          255.255.255.255  DHCP    360 DHCP Request  - Transaction ID 0x3d9a8e71
369 34.281435    192.168.100.1    192.168.100.9    DHCP    590 DHCP ACK      - Transaction ID 0x3d9a8e71
490 40.656387    192.168.100.9    192.168.100.1    DHCP    348 DHCP Request  - Transaction ID 0xacd8586e
496 40.968047    192.168.100.1    192.168.100.9    DHCP    590 DHCP ACK      - Transaction ID 0xacd8586e
745 43.094268    0.0.0.0          255.255.255.255  DHCP    342 DHCP Request  - Transaction ID 0xc861fd8c
```

Figure 4: The filtered DHCP packets

We can see the packet sequence Discover, Offer, Request, Acknowledgment.

1.2.2 Questions

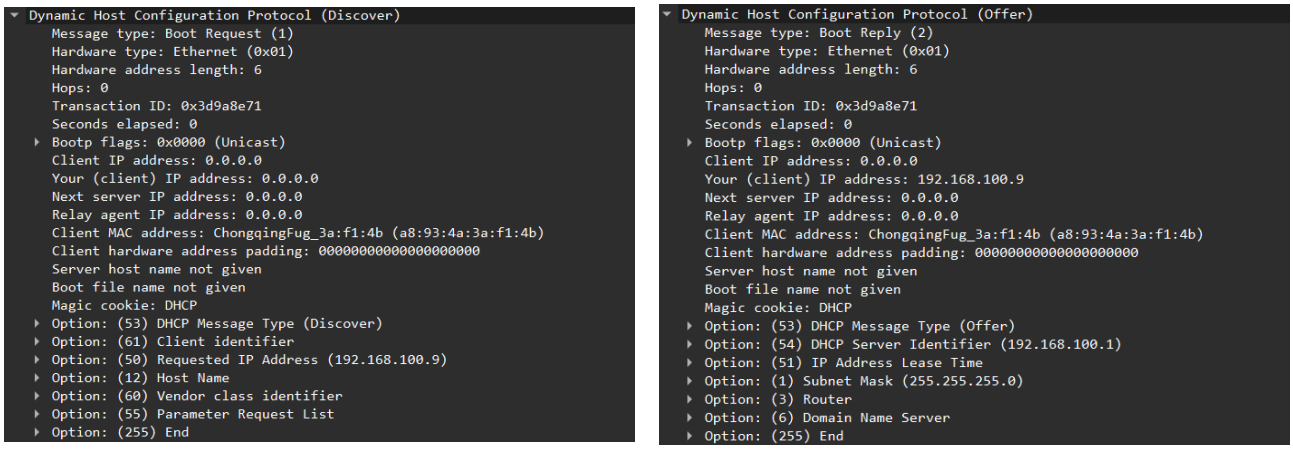


Figure 5: DHCP Discover and Offer packet details

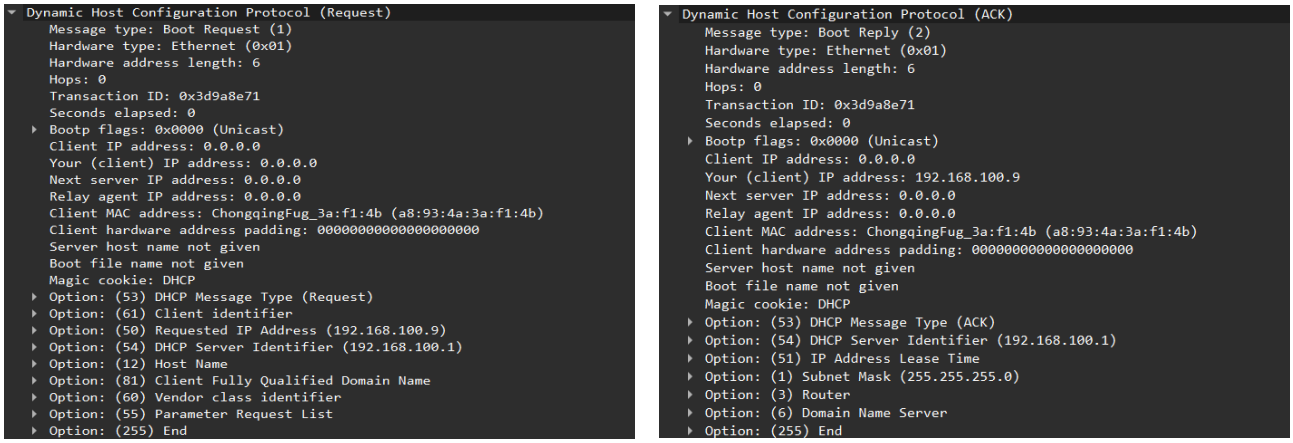


Figure 6: DHCP Request and Acknowledgment packet details

1. What is the MAC address and IP address of your machine as shown in the DHCP packets?

   - Discover:

     MAC address: a8:93:4a:3a:f1:4b my (client) IP address: 0.0.0.0

   - Offer:

     MAC address: a8:93:4a:3a:f1:4b my (client) IP address: 192.168.100.9

   - Request:

     MAC address: a8:93:4a:3a:f1:4b my (client) IP address: 0.0.0.0

   - ACK:

     MAC address: a8:93:4a:3a:f1:4b my (client) IP address: 192.168.100.9

2. Explain the purpose of the packet (e.g., DHCP Discover, Offer, Request, or Acknowledgment).

   The client sends a DHCP Discover packet to find available DHCP servers on the network. It's a broadcast packet 255.255.255.255 since the client doesn't have an IP address yet. Then the DHCP servers that receive the Discover respond with a DHCP Offer packet. This packet includes an available IP address, subnet mask, gateway, lease time, and possibly other info. The client responds to the offer by sending a DHCP Request packet, telling the chosen DHCP server that it wants to use the offered IP address. This

is also broadcast, so all DHCP servers see it. The chosen DHCP server finalizes the process by sending a DHCP ACK packet to the client. This confirms the IP address assignment and provides configuration details.

1.2.3 Simulation Scenario: DHCP Server on Laptop via Mobile Hotspot

In this experiment I enabled the mobile hotspot of my laptop after beginning to capture packet on the interface "Local Area Connection* 2". Then I connected my phone to the hotspot and after a few seconds stopped capturing. The filtered DHCP packets are shown in the figure below.

No.	Time	Source	Destination	Protocol	Length Info
13	26.828564	0.0.0.0	255.255.255.255	DHCP	352 DHCP Discover - Transaction ID 0xed44ecf7
14	26.834361	192.168.137.1	192.168.137.130	DHCP	344 DHCP Offer - Transaction ID 0xed44ecf7
17	26.874996	0.0.0.0	255.255.255.255	DHCP	362 DHCP Request - Transaction ID 0xed44ecf7
18	26.881444	192.168.137.1	192.168.137.130	DHCP	344 DHCP ACK - Transaction ID 0xed44ecf7

Figure 7

1.2.4 Questions

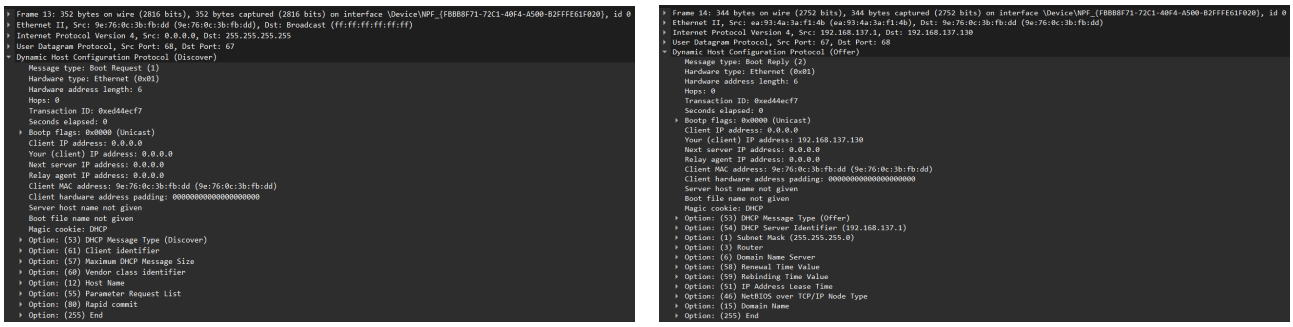


Figure 8: DHCP Discover and Offer packet details

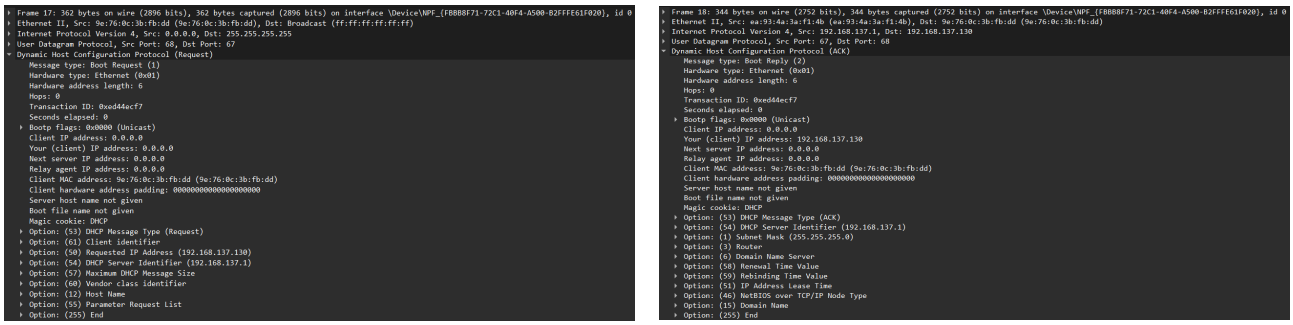


Figure 9: DHCP Request and Acknowledgment packet details

1. Which device is the DHCP server in this setup? What is its IP and MAC address?

   The DHCP server is my laptop since it's providing hotspot. As observed in the preceding figure, the server IP address is 192.168.137.1 and the server MAC address is ea:93:4a:3a:f1:4b.

2. Which device is the DHCP client? What IP address is assigned to it?

   The DHCP client is my phone because it is requesting IP. The client IP address and MAC address are 192.168.137.130 and 9e:76:0c:3b:fb:dd respectively.

3. **In the DHCP Discover packet:** What is the source MAC address? What is the destination MAC and IP address?

   The source MAC address is 9e:76:0c:3b:fb:dd. The destination MAC and IP addresses are ff:ff:ff:ff:ff:ff and 255.255.255.255 respectively because it is broadcast.

4. **In the DHCP Offer packet:** What IP address is offered to the phone? What information does the server provide in the DHCP Options (e.g., gateway, DNS)?

   The IP address offered and assigned to my phone is 192.168.137.130 . The info provided in the DHCP options are demonstrated in the following figure.

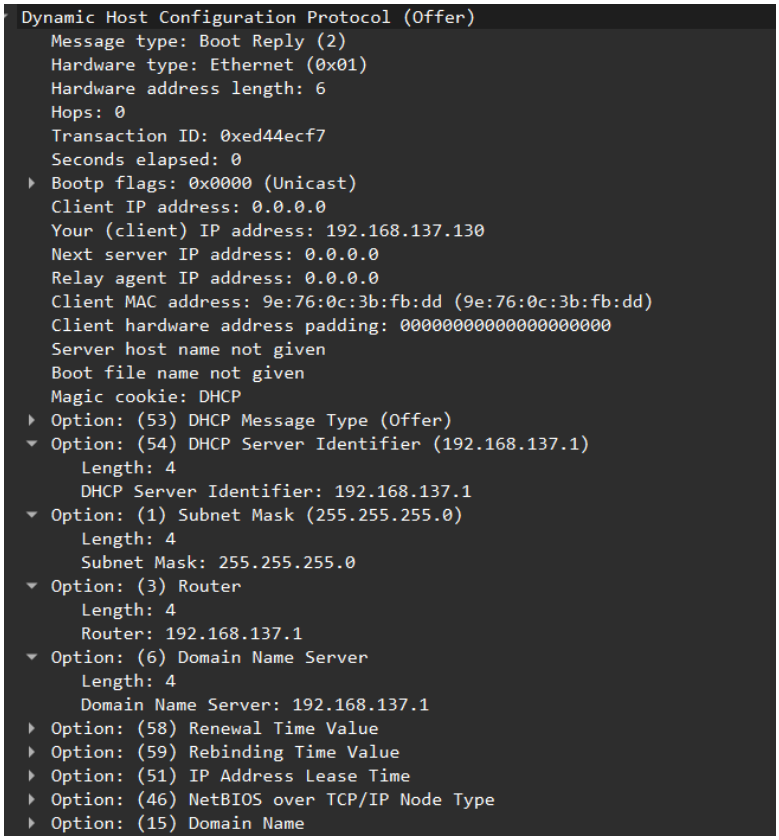


Figure 10: The provided information in DHCP options of DHCP offer packet

   The DHCP server provides the router/gateway address as 192.168.137.1 . The DHCP server provides the DNS address as 192.168.137.1 . The subnet mask offered is 255.255.255.0 . Other options are also provided such as lease time, renewal time, and rebind time.

5. What is the role of your laptop in this DHCP exchange? Is it acting like a NAT device too?

   My laptop is acting as a DHCP server because it is providing IP address assignments to my phone. It is also functioning as a mobile hotspot to share its network connection with my phone.

   Yes, the laptop is acting like a NAT (Network Address Translation) device because it provides internet access to my phone via the mobile hotspot. This process translates the private IP address of the phone (192.168.137.130) into the public IP address of my laptop's network interface, enabling my phone to access the internet.

6. Did you observe any difference between this exchange and the one in the earlier wired/wireless DHCP test? Explain briefly.

   The main difference is in the IP address range. The mobile hotspot assigns IP addresses from the 192.168.137.x range, while the Wi-Fi interface assigns addresses from the 192.168.100.x range. Also the Wi-Fi interface exchange includes a Requested IP Address in the DHCP Discover message, and the server responds with that specific IP in the Offer packet, while the mobile hotspot just offers a dynamically assigned IP address.

1.3 Analyze ARP Protocol

1.3.1 Simulation Scenario

I started capturing packets on the WiFi interface then cleared ARP cache using the command *arp -d ** in the command prompt. After a few seconds I stopped the capture and filtered the ARP packets.

No.	Time	Source	Destination	Protocol	Length	Info
8	3.978373	ChongqingFug_3a:f1:4b	Broadcast	ARP	42	Who has 192.168.54.175? Tell 192.168.54.251
9	3.998653	8e:97:3d:70:ac:dc	ChongqingFug_3a:f1:4b	ARP	42	192.168.54.175 is at 8e:97:3d:70:ac:dc
34	19.670254	8e:97:3d:70:ac:dc	ChongqingFug_3a:f1:4b	ARP	42	Who has 192.168.54.251? Tell 192.168.54.175
35	19.670269	ChongqingFug_3a:f1:4b	8e:97:3d:70:ac:dc	ARP	42	192.168.54.251 is at a8:93:4a:3a:f1:4b

Figure 11: The filtered arp packets

1.3.2 Questions

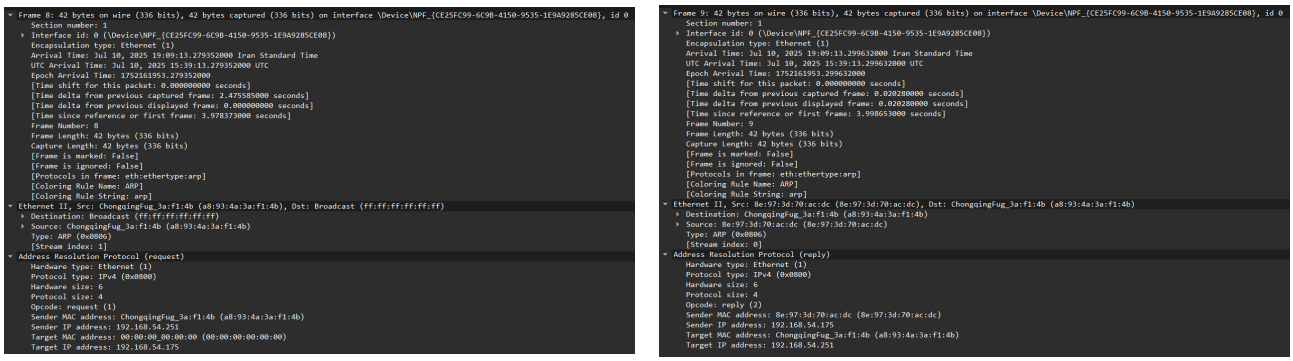


Figure 12: The first ARP request and ARP reply packet details

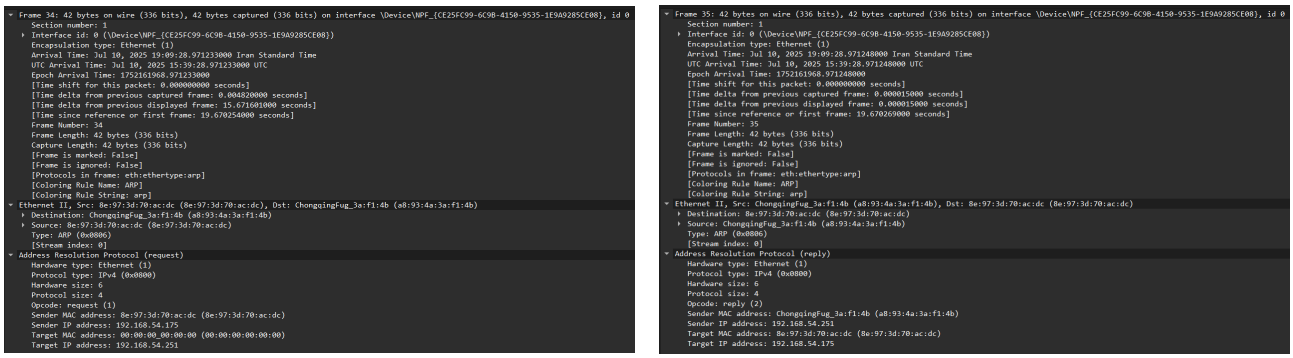


Figure 13: The second ARP request and ARP reply packet details

1. Identify the source and destination MAC and IP addresses.

   In the first ARP request (frame 8) source (sender) MAC and IP addresses are a8:93:4a:3a:f1:4b and 192.168.54.251 and destination (target) MAC and IP addresses are 00:00:00:00:00:00 (broadcast) and 192.168.54.175 respectively.

   In the first ARP reply (frame 9) source (sender) MAC and IP addresses are 8e:97:3d:70:ac:dc and 192.168.54.175 and destination (target) MAC and IP addresses are a8:93:4a:3a:f1:4b and 192.168.54.251 respectively.

   In the second ARP request (frame 34) source (sender) MAC and IP addresses are 8e:97:3d:70:ac:dc and 192.168.54.175 and destination (target) MAC and IP addresses are 00:00:00:00:00:00 (broadcast) and 192.168.54.251 respectively.

   In the second ARP reply (frame 35) source (sender) MAC and IP addresses are a8:93:4a:3a:f1:4b and 192.168.54.251 and destination (target) MAC and IP addresses are 8e:97:3d:70:ac:dc and 192.168.54.175 respectively.

2. Explain the packet's purpose (e.g., ARP request or reply).

   The ARP request is sent by a device when it needs to find the MAC address corresponding to an IP address on the local network.

   The ARP reply is sent by the device that has the requested IP address, responding with its MAC address.

3. Open the ARP section in the Packet Details window and describe its fields.

   - Hardware type:

     Ethernet (1), meaning the ARP is for resolving Ethernet MAC addresses.

   - Protocol type:

     IPv4 (0x0800), indicating that this ARP request/reply is related to IPv4.

   - Hardware size:

     6, which is the size of the MAC address (6 bytes).

   - Protocol size:

     4, which is the size of the IP address (4 bytes).

   - Opcode:

     1 for ARP request and 2 for ARP reply. The ARP request asks "Who has this IP?" while the ARP reply answers with the MAC address.

   - Sender MAC address:

     The MAC address of the device sending the ARP message.

   - Sender IP address:

     The IP address of the device sending the ARP message.

   - Target MAC address:

     The MAC address of the device being queried.

   - Target IP address:

     The IP address of the device being queried.

4. Why does the system need to use ARP before sending the first ping packet?

   The system needs to use ARP (Address Resolution Protocol) before sending a ping because ARP is necessary to resolve the MAC address associated with the destination IP address in the local network.

   A ping packet requires the MAC address of the destination device to be able to send the data link (Ethernet) frame.

5. Are ARP messages sent as broadcast or unicast? How can you tell from the packet capture?

   ARP requests are broadcasted because the sender does not know which device has the requested IP address.

   ARP replies are unicasted to the device that requested the information.

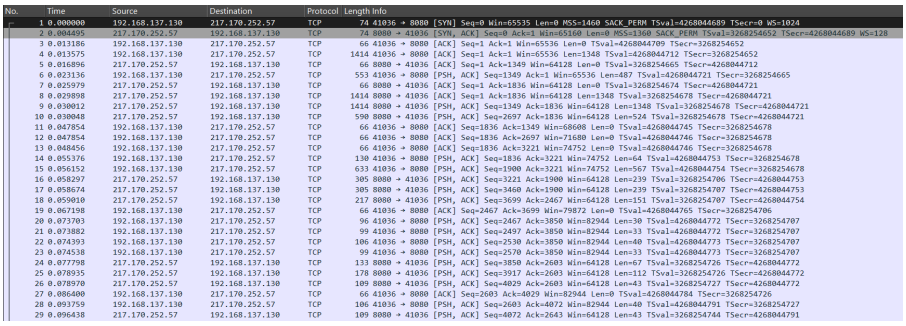
1.4 Analyze NAT Behavior Using Wireshark

1.4.1 Simulation Scenario

I captured packets using two instances of WireShark and then filtered the captured packets after the speed test process was over. Then I saved the selected packets in two .pcapng files named WiFi and HotSpot.

1.4.2 Questions

1. What is the IP address of your phone in the hotspot capture? What port did it use to initiate the connection?

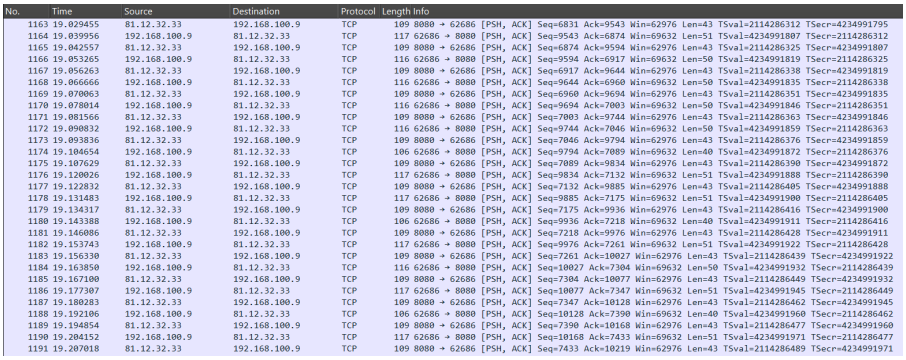


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.137.130	217.170.252.57	TCP	74	41036 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=4268044689 TSecr=0 WS=1024
2	0.004495	217.170.252.57	192.168.137.130	TCP	74	8080 → 41036 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1360 SACK_PERM TSval=3268254652 TSecr=4268044689 WS=128
3	0.013186	192.168.137.130	217.170.252.57	TCP	66	41036 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=4268044709 TSecr=3268254652
4	0.013575	192.168.137.130	217.170.252.57	TCP	1414	41036 → 8080 [ACK] Seq=1 Ack=1 Win=65536 Len=1348 TSval=4268044712 TSecr=3268254652
5	0.016896	217.170.252.57	192.168.137.130	TCP	66	8080 → 41036 [ACK] Seq=1 Ack=1349 Win=64128 Len=0 TSval=3268254665 TSecr=4268044712
6	0.023136	192.168.137.130	217.170.252.57	TCP	553	41036 → 8080 [PSH, ACK] Seq=1349 Ack=1 Win=65536 Len=487 TSval=4268044721 TSecr=3268254665
7	0.025979	217.170.252.57	192.168.137.130	TCP	66	8080 → 41036 [ACK] Seq=1 Ack=1836 Win=64128 Len=0 TSval=3268254674 TSecr=4268044721
8	0.029898	217.170.252.57	192.168.137.130	TCP	1414	8080 → 41036 [ACK] Seq=1 Ack=1836 Win=64128 Len=1348 TSval=3268254678 TSecr=4268044721
9	0.030012	217.170.252.57	192.168.137.130	TCP	1414	8080 → 41036 [PSH, ACK] Seq=1349 Ack=1836 Win=64128 Len=1348 TSval=3268254678 TSecr=4268044721
10	0.030048	217.170.252.57	192.168.137.130	TCP	590	8080 → 41036 [PSH, ACK] Seq=2697 Ack=1836 Win=64128 Len=524 TSval=3268254678 TSecr=4268044721
11	0.047854	192.168.137.130	217.170.252.57	TCP	66	41036 → 8080 [ACK] Seq=1836 Ack=1349 Win=68608 Len=0 TSval=4268044745 TSecr=3268254678
12	0.047854	192.168.137.130	217.170.252.57	TCP	66	41036 → 8080 [ACK] Seq=1836 Ack=2697 Win=71680 Len=0 TSval=4268044746 TSecr=3268254678
13	0.048456	192.168.137.130	217.170.252.57	TCP	66	41036 → 8080 [ACK] Seq=1836 Ack=3221 Win=74752 Len=0 TSval=4268044746 TSecr=3268254678
14	0.055376	192.168.137.130	217.170.252.57	TCP	130	41036 → 8080 [PSH, ACK] Seq=1836 Ack=3221 Win=74752 Len=64 TSval=4268044753 TSecr=3268254678
15	0.056152	192.168.137.130	217.170.252.57	TCP	633	41036 → 8080 [PSH, ACK] Seq=1900 Ack=3221 Win=74752 Len=567 TSval=4268044754 TSecr=3268254678
16	0.058297	217.170.252.57	192.168.137.130	TCP	305	8080 → 41036 [PSH, ACK] Seq=3221 Ack=1900 Win=64128 Len=239 TSval=3268254706 TSecr=4268044753
17	0.058674	217.170.252.57	192.168.137.130	TCP	305	8080 → 41036 [PSH, ACK] Seq=3460 Ack=1900 Win=64128 Len=239 TSval=3268254707 TSecr=4268044753
18	0.059010	217.170.252.57	192.168.137.130	TCP	217	8080 → 41036 [PSH, ACK] Seq=3699 Ack=2467 Win=64128 Len=151 TSval=3268254707 TSecr=4268044754
19	0.067198	192.168.137.130	217.170.252.57	TCP	66	41036 → 8080 [ACK] Seq=2467 Ack=3699 Win=79872 Len=0 TSval=4268044765 TSecr=3268254706
20	0.073703	192.168.137.130	217.170.252.57	TCP	96	41036 → 8080 [PSH, ACK] Seq=2467 Ack=3850 Win=82944 Len=30 TSval=4268044772 TSecr=3268254707
21	0.073882	192.168.137.130	217.170.252.57	TCP	99	41036 → 8080 [PSH, ACK] Seq=2497 Ack=3850 Win=82944 Len=33 TSval=4268044772 TSecr=3268254707
22	0.074393	192.168.137.130	217.170.252.57	TCP	106	41036 → 8080 [PSH, ACK] Seq=2530 Ack=3850 Win=82944 Len=40 TSval=4268044773 TSecr=3268254707
23	0.074538	192.168.137.130	217.170.252.57	TCP	99	41036 → 8080 [PSH, ACK] Seq=2570 Ack=3850 Win=82944 Len=33 TSval=4268044773 TSecr=3268254707
24	0.077798	217.170.252.57	192.168.137.130	TCP	133	8080 → 41036 [PSH, ACK] Seq=3850 Ack=2603 Win=64128 Len=67 TSval=3268254726 TSecr=4268044772
25	0.078935	217.170.252.57	192.168.137.130	TCP	178	8080 → 41036 [PSH, ACK] Seq=3917 Ack=2603 Win=64128 Len=112 TSval=3268254726 TSecr=4268044772
26	0.078970	217.170.252.57	192.168.137.130	TCP	109	8080 → 41036 [PSH, ACK] Seq=4029 Ack=2603 Win=64128 Len=43 TSval=3268254727 TSecr=4268044772
27	0.086400	192.168.137.130	217.170.252.57	TCP	66	41036 → 8080 [ACK] Seq=2603 Ack=4029 Win=82944 Len=0 TSval=4268044784 TSecr=3268254726
28	0.093759	192.168.137.130	217.170.252.57	TCP	106	41036 → 8080 [PSH, ACK] Seq=2603 Ack=4072 Win=82944 Len=40 TSval=4268044791 TSecr=3268254727
29	0.096438	217.170.252.57	192.168.137.130	TCP	109	8080 → 41036 [PSH, ACK] Seq=4072 Ack=2643 Win=64128 Len=43 TSval=3268254744 TSecr=4268044791

Figure 14: The first few filtered packets in the HotSpot capture

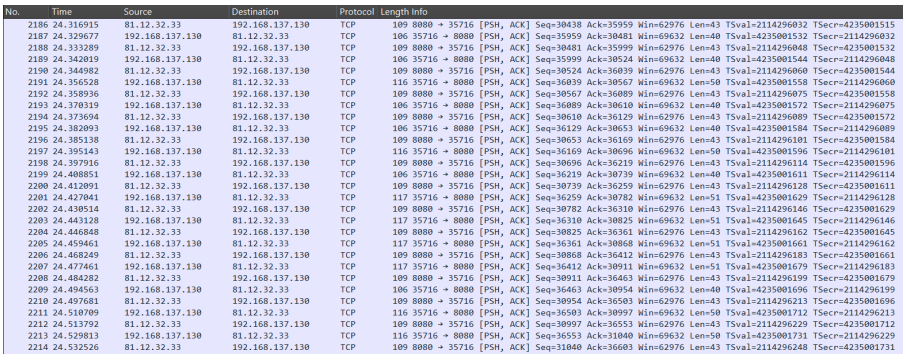
According to the first packet the IP address of my phone in the hotspot capture is 192.168.137.130 . The source port used to initiate the connection was 41036 based of the info section of the first packet presented in 14.

2. What is the destination IP and port (i.e., the server performing the speed test)?



No.	Time	Source	Destination	Protocol	Length	Info
1163	19.029455	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=6831 Ack=9543 Win=62976 Len=43 TSval=2114286312 TSecr=4234991795
1164	19.039956	192.168.100.9	81.12.32.33	TCP	117	62686 → 8080 [PSH, ACK] Seq=9543 Ack=6874 Win=69632 Len=51 TSval=4234991807 TSecr=2114286312
1165	19.042557	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=6874 Ack=9594 Win=62976 Len=43 TSval=2114286325 TSecr=4234991807
1166	19.053265	192.168.100.9	81.12.32.33	TCP	116	62686 → 8080 [PSH, ACK] Seq=9594 Ack=6917 Win=69632 Len=50 TSval=4234991819 TSecr=2114286325
1167	19.056263	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=6917 Ack=9644 Win=62976 Len=43 TSval=2114286338 TSecr=4234991819
1168	19.066666	192.168.100.9	81.12.32.33	TCP	116	62686 → 8080 [PSH, ACK] Seq=9644 Ack=6960 Win=69632 Len=50 TSval=4234991835 TSecr=2114286338
1169	19.070063	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=6960 Ack=9694 Win=62976 Len=43 TSval=2114286351 TSecr=4234991835
1170	19.078014	192.168.100.9	81.12.32.33	TCP	116	62686 → 8080 [PSH, ACK] Seq=9694 Ack=7003 Win=69632 Len=50 TSval=4234991846 TSecr=2114286351
1171	19.081566	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7003 Ack=9744 Win=62976 Len=43 TSval=2114286363 TSecr=4234991846
1172	19.090832	192.168.100.9	81.12.32.33	TCP	116	62686 → 8080 [PSH, ACK] Seq=9744 Ack=7046 Win=69632 Len=50 TSval=4234991859 TSecr=2114286363
1173	19.093836	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7046 Ack=9794 Win=62976 Len=43 TSval=2114286376 TSecr=4234991859
1174	19.104654	192.168.100.9	81.12.32.33	TCP	106	62686 → 8080 [PSH, ACK] Seq=9794 Ack=7089 Win=69632 Len=40 TSval=4234991872 TSecr=2114286376
1175	19.107629	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7089 Ack=9834 Win=62976 Len=43 TSval=2114286390 TSecr=4234991872
1176	19.120026	192.168.100.9	81.12.32.33	TCP	117	62686 → 8080 [PSH, ACK] Seq=9834 Ack=7132 Win=69632 Len=51 TSval=4234991888 TSecr=2114286390
1177	19.122832	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7132 Ack=9885 Win=62976 Len=43 TSval=2114286405 TSecr=4234991888
1178	19.131483	192.168.100.9	81.12.32.33	TCP	117	62686 → 8080 [PSH, ACK] Seq=9885 Ack=7175 Win=69632 Len=51 TSval=4234991900 TSecr=2114286405
1179	19.134317	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7175 Ack=9936 Win=62976 Len=43 TSval=2114286416 TSecr=4234991900
1180	19.143388	192.168.100.9	81.12.32.33	TCP	106	62686 → 8080 [PSH, ACK] Seq=9936 Ack=7218 Win=69632 Len=40 TSval=4234991911 TSecr=2114286416
1181	19.146086	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7218 Ack=9976 Win=62976 Len=43 TSval=2114286428 TSecr=4234991911
1182	19.153743	192.168.100.9	81.12.32.33	TCP	117	62686 → 8080 [PSH, ACK] Seq=9976 Ack=7261 Win=69632 Len=51 TSval=4234991922 TSecr=2114286428
1183	19.156330	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7261 Ack=10027 Win=62976 Len=43 TSval=2114286439 TSecr=4234991922
1184	19.163850	192.168.100.9	81.12.32.33	TCP	116	62686 → 8080 [PSH, ACK] Seq=10027 Ack=7304 Win=69632 Len=50 TSval=4234991932 TSecr=2114286439
1185	19.167100	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7304 Ack=10077 Win=62976 Len=43 TSval=2114286449 TSecr=4234991932
1186	19.177307	192.168.100.9	81.12.32.33	TCP	117	62686 → 8080 [PSH, ACK] Seq=10077 Ack=7347 Win=69632 Len=51 TSval=4234991945 TSecr=2114286449
1187	19.180283	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7347 Ack=10128 Win=62976 Len=43 TSval=2114286462 TSecr=4234991945
1188	19.192106	192.168.100.9	81.12.32.33	TCP	106	62686 → 8080 [PSH, ACK] Seq=10128 Ack=7390 Win=69632 Len=40 TSval=4234991960 TSecr=2114286462
1189	19.194854	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7390 Ack=10168 Win=62976 Len=43 TSval=2114286477 TSecr=4234991960
1190	19.204152	192.168.100.9	81.12.32.33	TCP	117	62686 → 8080 [PSH, ACK] Seq=10168 Ack=7433 Win=69632 Len=51 TSval=4234991971 TSecr=2114286477
1191	19.207018	81.12.32.33	192.168.100.9	TCP	109	8080 → 62686 [PSH, ACK] Seq=7433 Ack=10219 Win=62976 Len=43 TSval=2114286489 TSecr=4234991971

Figure 15: Some test packets in WiFi capture



No.	Time	Source	Destination	Protocol	Length	Info
2186	24.316915	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30438 Ack=35959 Win=62976 Len=43 TSval=2114296032 TSecr=4235001515
2187	24.329677	192.168.137.130	81.12.32.33	TCP	106	35716 → 8080 [PSH, ACK] Seq=35959 Ack=30481 Win=69632 Len=40 TSval=4235001532 TSecr=2114296032
2188	24.333289	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30481 Ack=35999 Win=62976 Len=43 TSval=2114296048 TSecr=4235001532
2189	24.342019	192.168.137.130	81.12.32.33	TCP	106	35716 → 8080 [PSH, ACK] Seq=35999 Ack=30524 Win=69632 Len=40 TSval=4235001544 TSecr=2114296048
2190	24.344982	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30524 Ack=36039 Win=62976 Len=43 TSval=2114296060 TSecr=4235001544
2191	24.356528	192.168.137.130	81.12.32.33	TCP	116	35716 → 8080 [PSH, ACK] Seq=36039 Ack=30567 Win=69632 Len=50 TSval=4235001558 TSecr=2114296060
2192	24.358936	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30567 Ack=36089 Win=62976 Len=43 TSval=2114296075 TSecr=4235001558
2193	24.370319	192.168.137.130	81.12.32.33	TCP	106	35716 → 8080 [PSH, ACK] Seq=36089 Ack=30610 Win=69632 Len=40 TSval=4235001572 TSecr=2114296075
2194	24.373694	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30610 Ack=36129 Win=62976 Len=43 TSval=2114296089 TSecr=4235001572
2195	24.382093	192.168.137.130	81.12.32.33	TCP	106	35716 → 8080 [PSH, ACK] Seq=36129 Ack=30653 Win=69632 Len=40 TSval=4235001584 TSecr=2114296089
2196	24.385138	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30653 Ack=36169 Win=62976 Len=43 TSval=2114296101 TSecr=4235001584
2197	24.395143	192.168.137.130	81.12.32.33	TCP	116	35716 → 8080 [PSH, ACK] Seq=36169 Ack=30696 Win=69632 Len=50 TSval=4235001596 TSecr=2114296101
2198	24.397916	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30696 Ack=36219 Win=62976 Len=43 TSval=2114296114 TSecr=4235001596
2199	24.408851	192.168.137.130	81.12.32.33	TCP	106	35716 → 8080 [PSH, ACK] Seq=36219 Ack=30739 Win=69632 Len=40 TSval=4235001611 TSecr=2114296114
2200	24.412091	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30739 Ack=36259 Win=62976 Len=43 TSval=2114296128 TSecr=4235001611
2201	24.427041	192.168.137.130	81.12.32.33	TCP	117	35716 → 8080 [PSH, ACK] Seq=36259 Ack=30782 Win=69632 Len=51 TSval=4235001629 TSecr=2114296128
2202	24.430514	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30782 Ack=36310 Win=62976 Len=43 TSval=2114296146 TSecr=4235001629
2203	24.443128	192.168.137.130	81.12.32.33	TCP	117	35716 → 8080 [PSH, ACK] Seq=36310 Ack=30825 Win=69632 Len=51 TSval=4235001645 TSecr=2114296146
2204	24.446848	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30825 Ack=36361 Win=62976 Len=43 TSval=2114296162 TSecr=4235001645
2205	24.459461	192.168.137.130	81.12.32.33	TCP	117	35716 → 8080 [PSH, ACK] Seq=36361 Ack=30868 Win=69632 Len=51 TSval=4235001661 TSecr=2114296162
2206	24.468249	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30868 Ack=36412 Win=62976 Len=43 TSval=2114296183 TSecr=4235001661
2207	24.477461	192.168.137.130	81.12.32.33	TCP	117	35716 → 8080 [PSH, ACK] Seq=36412 Ack=30911 Win=69632 Len=51 TSval=4235001679 TSecr=2114296183
2208	24.484282	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30911 Ack=36463 Win=62976 Len=43 TSval=2114296199 TSecr=4235001679
2209	24.494563	192.168.137.130	81.12.32.33	TCP	106	35716 → 8080 [PSH, ACK] Seq=36463 Ack=30954 Win=69632 Len=40 TSval=4235001696 TSecr=2114296199
2210	24.497681	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30954 Ack=36503 Win=62976 Len=43 TSval=2114296213 TSecr=4235001696
2211	24.510709	192.168.137.130	81.12.32.33	TCP	116	35716 → 8080 [PSH, ACK] Seq=36503 Ack=30997 Win=69632 Len=50 TSval=4235001712 TSecr=2114296213
2212	24.513792	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=30997 Ack=36553 Win=62976 Len=43 TSval=2114296229 TSecr=4235001712
2213	24.529813	192.168.137.130	81.12.32.33	TCP	116	35716 → 8080 [PSH, ACK] Seq=36553 Ack=31040 Win=69632 Len=50 TSval=4235001731 TSecr=2114296229
2214	24.532526	81.12.32.33	192.168.137.130	TCP	109	8080 → 35716 [PSH, ACK] Seq=31040 Ack=36603 Win=62976 Len=43 TSval=2114296248 TSecr=4235001731

Figure 16: Some test packets in HotSpot capture

Based on the figures presented above, the IP address of the server performing the speed test is 81.12.32.33 . The destination port is 8080.

3. Compare the IP addresses and ports in both captures. Explain how NAT translated them.

HotSpot capture:

phone IP: 192.168.137.130 phone Port: 41036
server IP: 81.12.32.33 server Port: 8080

WiFi Capture:

phone IP: 192.168.100.9 phone Port: 62532
server IP: 81.12.32.33 server Port: 8080

NAT translated my phone's private IP and port 192.168.137.130:41036 into my laptop's IP and a new port 192.168.100.9:62532 to enable internet access. Since private IPs aren't routable on the internet, the laptop changed the source address and kept a mapping in its NAT table. When the server responded, NAT reversed the translation and delivered the reply back to the phone, ensuring proper communication between the device and the server.

4. Why is NAT needed in this case? Could your phone directly access the internet without the translation?

   NAT is essential in this scenario because the phone's private IP is not routable on the public internet, and the laptop must act as a gateway to translate the phone's traffic.
   Without NAT, my phone could not communicate with the speed test server, as its private IP would prevent proper routing and response handling.

1.5 Analyze Traceroute Using ICMP

1.5.1 Simulation Scenario

I started capturing traffic on the WiFi interface and then ran the traceroute command *tracert 8.8.8.8* to Google DNS. After a few seconds the traceroute completed and I stopped the capture. The filtered ICMP packets are demonstrated in the following figures.

```
C:\Users\juve2>tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  1     1 ms    <1 ms     1 ms  192.168.100.1
  2     4 ms     3 ms     5 ms  100.82.0.1
  3     3 ms     2 ms     2 ms  172.30.101.209
  4     8 ms     7 ms     9 ms  172.30.102.193
  5     3 ms     2 ms     3 ms  172.30.102.122
  6     3 ms     3 ms     3 ms  172.19.18.89
  7     3 ms     4 ms     2 ms  172.19.17.73
  8     7 ms     *        4 ms  172.19.17.41
  9     7 ms     3 ms     4 ms  10.202.6.194
 10     6 ms     4 ms     4 ms  10.21.211.10
 11     5 ms     4 ms    11 ms  10.21.21.10
 12     *        *       32 ms  213.202.4.172
 13    32 ms     *       29 ms  213.202.5.239
 14    31 ms    30 ms    37 ms  216.239.48.87
 15    30 ms    36 ms    28 ms  192.178.250.97
 16    30 ms    29 ms    29 ms  dns.google [8.8.8.8]

Trace complete.
```

Figure 17: Traceroute to Google result

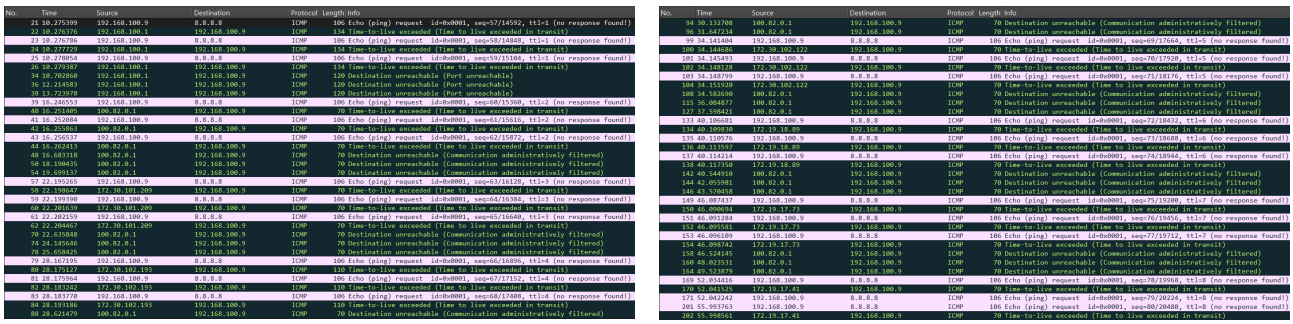


Figure 18: The ICMP packets

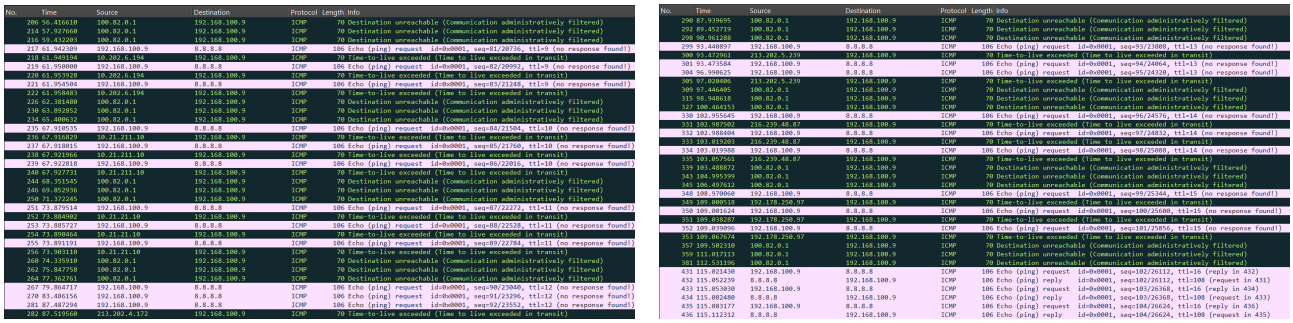


Figure 19: The ICMP packets

1.5.2 Questions

1. How many hops did it take to reach 8.8.8.8? List the IP addresses of any 3 hops.

   According to the figures in the previous page, the total number of hops took to reach the destination was 16 hops.

   The hops respectively:

   $$192.168.100.9 \longrightarrow 192.168.100.1 \longrightarrow 100.82.0.1 \longrightarrow 172.30.101.209 \longrightarrow 172.30.102.193 \longrightarrow 172.30.102.122 \longrightarrow 172.19.18.89 \longrightarrow 172.19.17.73 \longrightarrow 172.19.17.41 \longrightarrow 10.202.6.194 \longrightarrow 10.21.211.10 \longrightarrow 10.21.21.10 \longrightarrow 213.202.5.239 \longrightarrow 216.239.48.87 \longrightarrow 192.178.250.97 \longrightarrow 8.8.8.8$$

2. What type of ICMP messages were generated by intermediate routers?

   The ICMP message type generated by intermediate routers is Time to live exceeded (Type 11 Code 0).

   These time exceeded messages are generated when the Time to Live (TTL) of a packet reaches zero before it reaches its destination. This shows that the packet has been forwarded through multiple routers but has not yet reached the destination.

3. Pick one Time Exceeded packet and describe its source IP, destination IP, and TTL value in the IP header.

   According to this figure,

   Source IP address: 192.168.100.1

   Destination IP address: 192.168.100.9

   TTL: 64

4. Why do intermediate routers generate Time Exceeded messages?

   If the TTL reaches zero before the packet reaches the destination, the router sends an ICMP Time Exceeded message back to the source, notifying that the packet was discarded due to TTL expiration.

   This mechanism is used to track the path a packet takes across the network by setting the TTL value and receiving ICMP responses from each router.

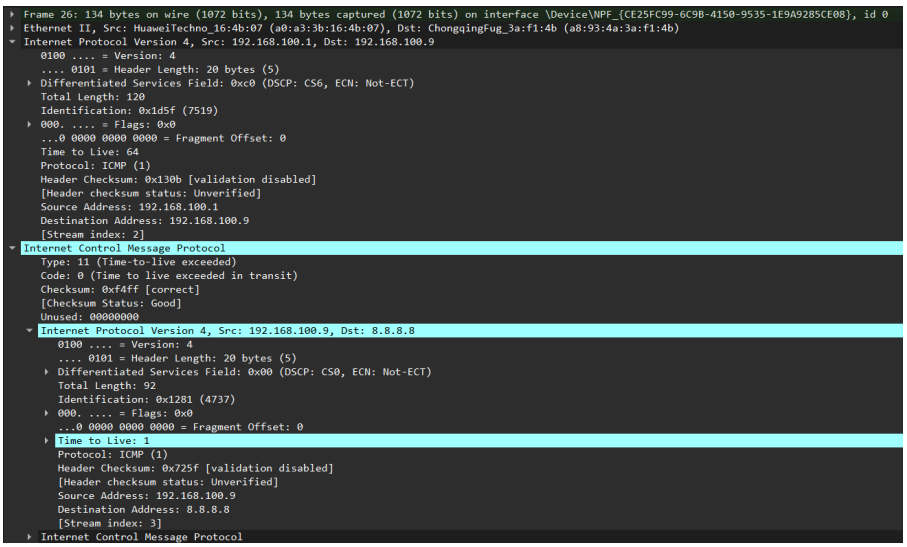


Figure 20: The details of the first Time Exceeded packet

5. What happens to the TTL value in each step of the traceroute?

   TTL starts from a low value (here 1) and increases with each subsequent packet.

   In the first hop the TTL is set to 1, so the first router will discard the packet, sending back an ICMP Time Exceeded message.

   In the next hop TTL is incremented to 2, allowing the packet to reach the second router, which decreases the TTL to 1. This router then responds with a Time Exceeded message.

   This process continues, increasing the TTL by 1 each time until the packet eventually reaches the destination or the maximum number of hops is reached(here 30).

6. Is the final destination's response different from the intermediate hops? What ICMP type/code does it return?

   Yes the final destination's response is an Echo reply with Type 0 and Code 0.

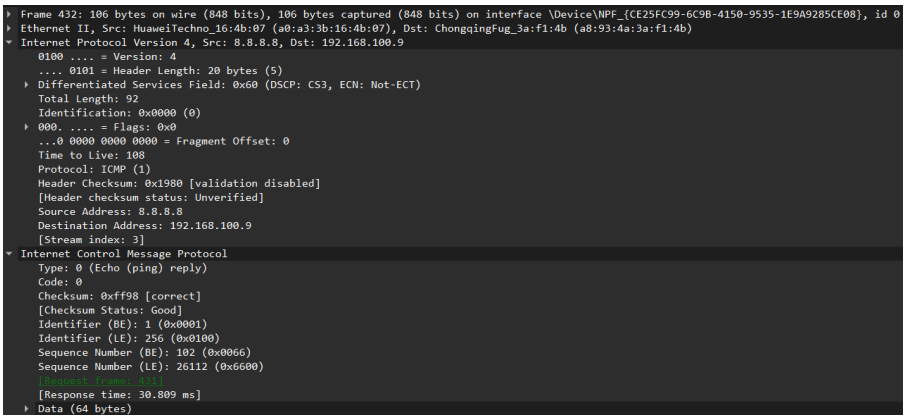


Figure 21: The final destination response packet details

1.6 Analyze IP Fragmentation

1.6.1 Simulation Scenario

1.6.2 Questions

1. How many fragments were generated for one large ping request?

   As we can see each large ping request is fragmented into two fragments.

```
C:\Users\juve2>ping -l 2000 192.168.100.1

Pinging 192.168.100.1 with 2000 bytes of data:
Reply from 192.168.100.1: bytes=2000 time=4ms TTL=64
Reply from 192.168.100.1: bytes=2000 time=4ms TTL=64
Reply from 192.168.100.1: bytes=2000 time=4ms TTL=64
Reply from 192.168.100.1: bytes=2000 time=5ms TTL=64

Ping statistics for 192.168.100.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 4ms, Maximum = 5ms, Average = 4ms
```

Figure 22: Ping request result in CMD

No.	Time	Source	Destination	Protocol	Length	Info
58	5.752469	192.168.100.9	192.168.100.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=2f7e) [Reassembled in #59]
59	5.752469	192.168.100.9	192.168.100.1	ICMP	562	Echo (ping) request id=0x0001, seq=121/30976, ttl=128 (reply in 61)
60	5.755962	192.168.100.1	192.168.100.9	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=4fa5) [Reassembled in #61]
61	5.756836	192.168.100.1	192.168.100.9	ICMP	562	Echo (ping) reply id=0x0001, seq=121/30976, ttl=64 (request in 59)
66	6.764478	192.168.100.9	192.168.100.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=2f7f) [Reassembled in #67]
67	6.764478	192.168.100.9	192.168.100.1	ICMP	562	Echo (ping) request id=0x0001, seq=122/31232, ttl=128 (reply in 69)
68	6.767897	192.168.100.1	192.168.100.9	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=4fe5) [Reassembled in #69]
69	6.768801	192.168.100.1	192.168.100.9	ICMP	562	Echo (ping) reply id=0x0001, seq=122/31232, ttl=64 (request in 67)
72	7.781070	192.168.100.9	192.168.100.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=2f80) [Reassembled in #73]
73	7.781070	192.168.100.9	192.168.100.1	ICMP	562	Echo (ping) request id=0x0001, seq=123/31488, ttl=128 (reply in 75)
74	7.784880	192.168.100.1	192.168.100.9	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=4fea) [Reassembled in #75]
75	7.785779	192.168.100.1	192.168.100.9	ICMP	562	Echo (ping) reply id=0x0001, seq=123/31488, ttl=64 (request in 73)
76	8.797668	192.168.100.9	192.168.100.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=2f81) [Reassembled in #77]
77	8.797668	192.168.100.9	192.168.100.1	ICMP	562	Echo (ping) request id=0x0001, seq=124/31744, ttl=128 (reply in 79)
78	8.801927	192.168.100.1	192.168.100.9	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=504d) [Reassembled in #79]
79	8.802824	192.168.100.1	192.168.100.9	ICMP	562	Echo (ping) reply id=0x0001, seq=124/31744, ttl=64 (request in 77)

Figure 23: The filtered packets

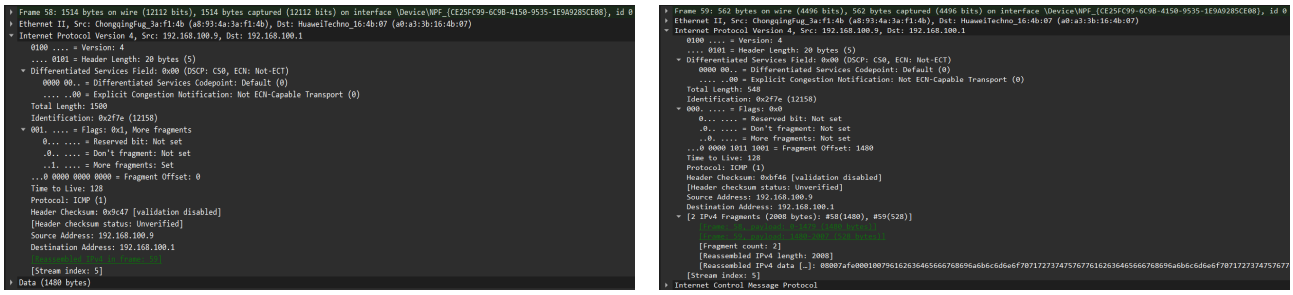


Figure 24: The two fragments of the first large ping request

2. What is the total length of each fragment? What is the offset of each?

   As observed above, the total length of the first fragment is 1500 bytes and the second fragment 548 bytes.

   offset of the first fragment is 0 and the second fragment 1480.

3. What values in the IP header indicate that a packet is fragmented?

   The flag in the first fragment is set to 0x1 which means that MF is set and the packet is fragmented and more fragments are expected. The offset in the second fragment is a number greater than 0 (1480) which also indicated that the packet is fragmented.

4. What is the role of the ”More Fragments” (MF) flag and Fragment Offset?

   The MF flag is used to indicate if the fragment is the last one. If MF = 1, it means there are more fragments to come. If MF = 0, it signifies that this is the last fragment in the packet.

   The fragment offset tells the receiving system where the current fragment belongs in the reassembly of the original datagram. It is measured in bytes, so an offset of 1480 means the fragment starts 1480 bytes into the original message.

5. If one fragment is lost during transmission, will the full datagram be delivered?

   No. If one fragment is lost, the receiving device cannot reassemble the full datagram because it is missing

a part of the data. The destination device will normally request the missing fragment or drop the entire datagram if it is incomplete.

6. What is the Identification field used for in fragmented IP packets?

   The Identification field is used to group fragments that belong to the same original datagram.

   As we can see in 37 both fragments have the Identification 0x2f7e.

7. Are fragmented packets reassembled by routers or by the destination device? Explain briefly.

   Reassembly of fragmented packets is done by the destination device.

   Routers do not reassemble fragmented packets; they only forward the fragments to the destination.

   The destination device uses the Identification field and Fragment Offset to reassemble the fragments into the original datagram.

1.6.3 Simulation Scenario

I repeated the previous scenario for DNS servers 8.8.8.8 and 1.1.1.1 . The results are demonstrated in the following figures.

8.8.8.8

```
C:\Users\juve2>ping -l 2000 8.8.8.8

Pinging 8.8.8.8 with 2000 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Figure 25: Ping request result in CMD

No.	Time	Source	Destination	Protocol	Length	Info
19	8.391293	192.168.100.9	8.8.8.8	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=12b3)
20	8.391293	192.168.100.9	8.8.8.8	ICMP	562	Echo (ping) request id=0x0001, seq=125/32000, ttl=128 (no response found!)
29	13.267344	192.168.100.9	8.8.8.8	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=12b4)
30	13.267344	192.168.100.9	8.8.8.8	ICMP	562	Echo (ping) request id=0x0001, seq=126/32256, ttl=128 (no response found!)
41	18.269359	192.168.100.9	8.8.8.8	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=12b5)
42	18.269359	192.168.100.9	8.8.8.8	ICMP	562	Echo (ping) request id=0x0001, seq=127/32512, ttl=128 (no response found!)
61	23.272786	192.168.100.9	8.8.8.8	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=12b6)
62	23.272786	192.168.100.9	8.8.8.8	ICMP	562	Echo (ping) request id=0x0001, seq=128/32768, ttl=128 (no response found!)

Figure 26: The filtered packets

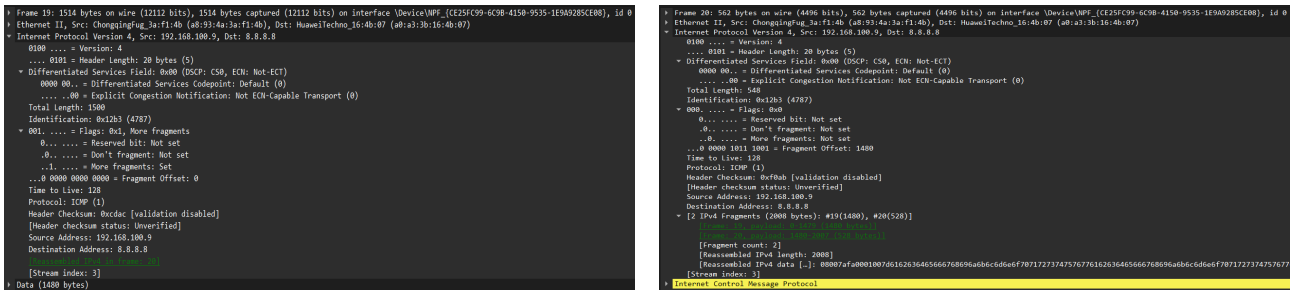


Figure 27: The two fragments of the first large ping request

1.1.1.1

```
C:\Users\juve2>ping -l 2000 1.1.1.1

Pinging 1.1.1.1 with 2000 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 1.1.1.1:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Figure 28: Ping request result in CMD

No.	Time	Source	Destination	Protocol	Length	Info
	16 4.373858	192.168.100.9	1.1.1.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=679f) [Reassembled in #17]
•	17 4.373858	192.168.100.9	1.1.1.1	ICMP	562	Echo (ping) request id=0x0001, seq=129/33024, ttl=128 (no response found!)
	35 9.008736	192.168.100.9	1.1.1.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=67a0)
	36 9.008736	192.168.100.9	1.1.1.1	ICMP	562	Echo (ping) request id=0x0001, seq=130/33280, ttl=128 (no response found!)
	39 14.012299	192.168.100.9	1.1.1.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=67a1)
	40 14.012299	192.168.100.9	1.1.1.1	ICMP	562	Echo (ping) request id=0x0001, seq=131/33536, ttl=128 (no response found!)
	50 19.015259	192.168.100.9	1.1.1.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=67a2)
	51 19.015259	192.168.100.9	1.1.1.1	ICMP	562	Echo (ping) request id=0x0001, seq=132/33792, ttl=128 (no response found!)

Figure 29: The filtered packets

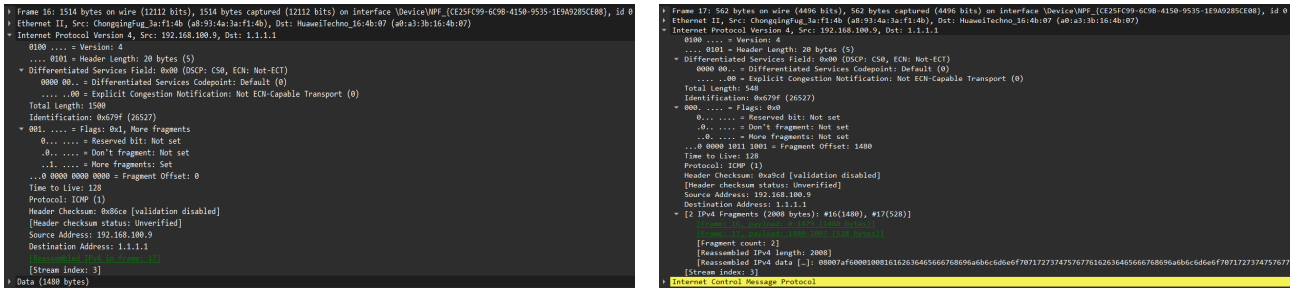


Figure 30: The two fragments of the first large ping request

As we can see the packets in this scenario are also fragmented into two fragments. All of the requests for both DNS servers ended up in a time out.

2 Ad-hoc On-Demand Distance Vector Routing

2.1 Simulation Setup

- **Application Parameters:**
  - Run a *PingApp* on each source node to send packets to the destination.
  - Configure the packet interval to **1 second**.
  - Log ping replies and packet loss for analysis.

  In the following figures the codes used in *.ini* and *.ned* files are provided.

```
 7
 8 *.source1.numApps = 1
 9 *.source1.app[0].typename = "PingApp"
10 *.source1.app[0].destAddr = "destination"
11 *.source1.app[0].printPing = true
12 *.source1.app[0].sendInterval = 1s
```

Figure 31: The code used in the *.ini* file

This code runs a **PingApp** on the source and configures the packet interval to 1s.

```
parameters:
    @display("bgb=1006.008,801.19196");
    @statistic[numRcvdPk](source=count(source1.app[0].pingRxSeq); record=figure; targetFigure=numRcvdPkCounter; checkSignals=false);
    @figure[numRcvdPkCounter](type=counter; transform=scale(0.8); pos=80,300; label="Received ping replies"; decimalPlaces=4);
    @statistic[numLost](source=count(source1.app[0].numLost); record=figure; targetFigure=numLostPkCounter; checkSignals=false);
    @figure[numLostPkCounter](type=counter; transform=scale(0.8); pos=80,400; label="Pings lost"; decimalPlaces=4);
```

Figure 32: The code used in the *.ned* file

This part of the code is responsible for logging ping replies and packet loss.

- **Mobility Configuration:**
  - Use *LinearMobility* for *Router1* and *Router2*.
  - Configure the mobility constraint area to limit node movement vertically with a constant speed (**10 m/s**).
  - Use the following constraints:
    * *.aodvRouter1.mobility.constraintAreaMinY = 50m
    * *.aodvRouter1.mobility.constraintAreaMaxY = 350m
    * *.aodvRouter2.mobility.constraintAreaMinY = 50m
    * *.aodvRouter2.mobility.constraintAreaMaxY = 350m

```
13
14 *.aodvRouter1.mobility.typename = "LinearMobility"
15 *.aodvRouter2.mobility.typename = "LinearMobility"
16 *.aodvRouter1.mobility.speed = 10mps
17 *.aodvRouter2.mobility.speed = 10mps
18
19 *.aodvRouter1.mobility.constraintAreaMinY = 50m
20 *.aodvRouter1.mobility.constraintAreaMaxY = 350m
21 *.aodvRouter2.mobility.constraintAreaMinY = 50m
22 *.aodvRouter2.mobility.constraintAreaMaxY = 350m
23 *.aodvRouter1.mobility.initialMovementHeading = 90deg
24 *.aodvRouter2.mobility.initialMovementHeading = 90deg
25
```

Figure 33: The code used in the *.ini* file

The code in the preceding figure is used to set *LinearMobility* to the needed routers and also configure the mobility in a way that makes sure the routers move vertically with a constant speed of 10 m/s.

We can also wee that the proper constraints are set for the mobility of routers 1 and 2.

- **Communication Range:**
  - Set the transmitter power of all nodes to **1 mW**.
  - Display communication ranges for all nodes to visualize network connectivity.

```
25
26 **.transmitter.power = 1mW
27 **.displayCommunicationRange = true
28
```

Figure 34: The code used in the *.ini* file

This part of the code sets the transmitter power of all the nodes to 1 mW and makes sure the communication range of each node is visible in the simulation.

- **Visualizer Configuration:**
  - Enable visualization of routing tables, network paths, and data links to better understand protocol dynamics.
  - Include counters to record the number of ping replies received and packets lost for each source.
  - Visualize these metrics during the simulation for real-time monitoring.

```
28
29 *.*.routingApp.typename = "Aodv"
30 *.visualizer.*.numDataLinkVisualizers = 5
31 *.visualizer.*.dataLinkVisualizer[*].activityLevel = "peer"
32 *.visualizer.*.dataLinkVisualizer[*].displayLinks = true
33 *.visualizer.*.dataLinkVisualizer[0].packetFilter = "aodv::Rreq"
34 *.visualizer.*.dataLinkVisualizer[1].packetFilter = "ping*"
35 *.visualizer.*.dataLinkVisualizer[1].*Color = "blue"
36 *.visualizer.*.dataLinkVisualizer[2].packetFilter = "aodv::Rrep"
37 *.visualizer.*.dataLinkVisualizer[2].*Color = "darkslategray"
38 *.visualizer.*.dataLinkVisualizer[3].packetFilter = "aodv::Rerr"
39 *.visualizer.*.dataLinkVisualizer[3].*Color = "red"
40 *.visualizer.*.dataLinkVisualizer[4].packetFilter = "Hello"
41 *.visualizer.*.dataLinkVisualizer[4].*Color = "green"
42 *.visualizer.*.dataLinkVisualizer[*].fadeOutMode = "simulationTime"
43
44 *.visualizer.*.routingTableVisualizer.displayLabels = false
45 *.visualizer.*.routingTableVisualizer.displayRoutesIndividually = false
46 *.visualizer.*.routingTableVisualizer.lineShift = 0
47 *.visualizer.*.networkRouteVisualizer.displayRoutes = true
48 *.visualizer.*.networkRouteVisualizer.packetFilter = "ping*"
49 *.visualizer.*.networkRouteVisualizer.fadeOutMode = "simulationTime"
50 *.visualizer.*.interfaceTableVisualizer.displayInterfaceTables = true
51 *.visualizer.*.interfaceTableVisualizer.format = "%a/%l"
52 *.visualizer.*.interfaceTableVisualizer.fadeOutMode = "simulationTime"
53
54 *.visualizer.*.routingTableVisualizer.displayRoutingTables = true
55 *.visualizer.*.routingTableVisualizer.destinationFilter = "destination"
56 *.visualizer.*.routingTableVisualizer.fadeOutMode = "simulationTime"
57
```

Figure 35: The code used in the *.ini* file

The code in the figure presented in the next page enables the visualization of the data link activity, routing tables and network routing. It is also used to set the routing protocol to AODV.

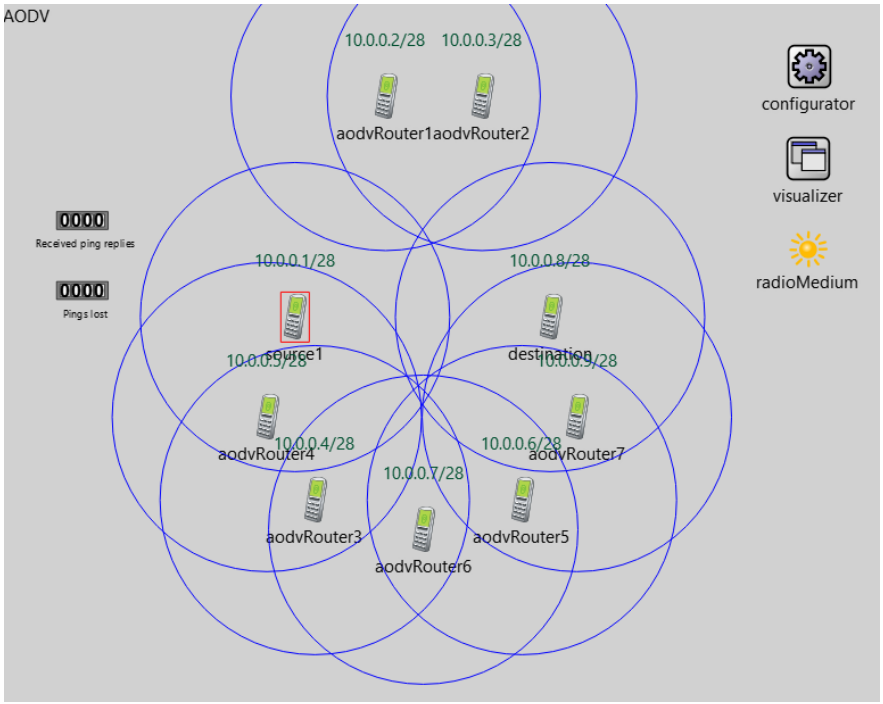
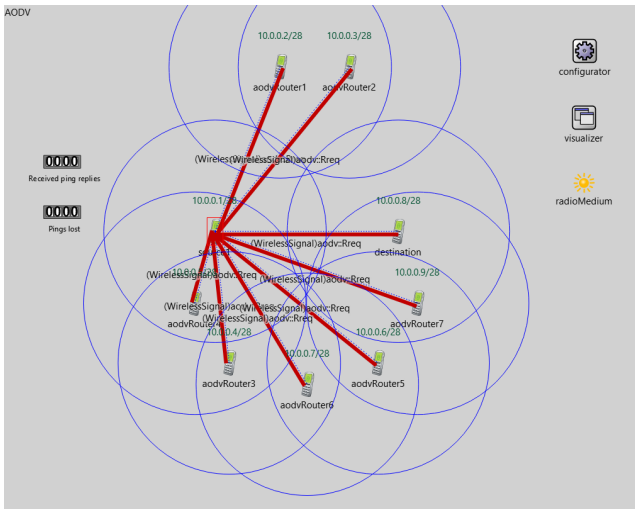


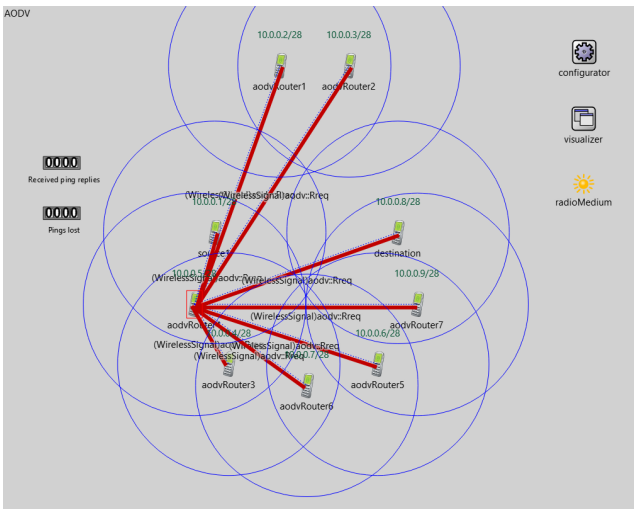
Figure 36: The description of the Network in the simulation

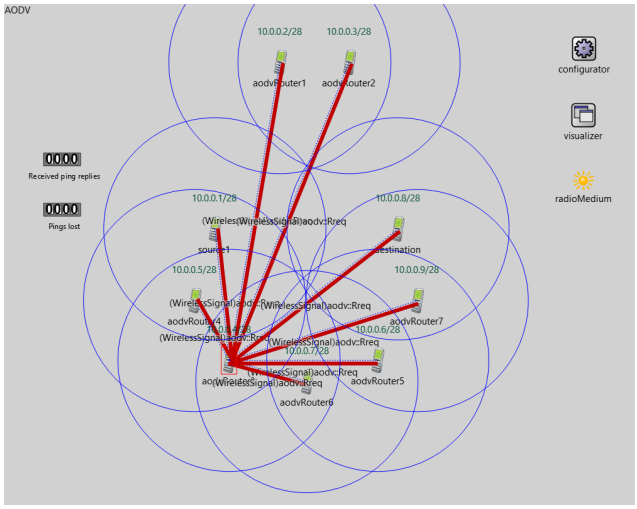
2.2 Questions

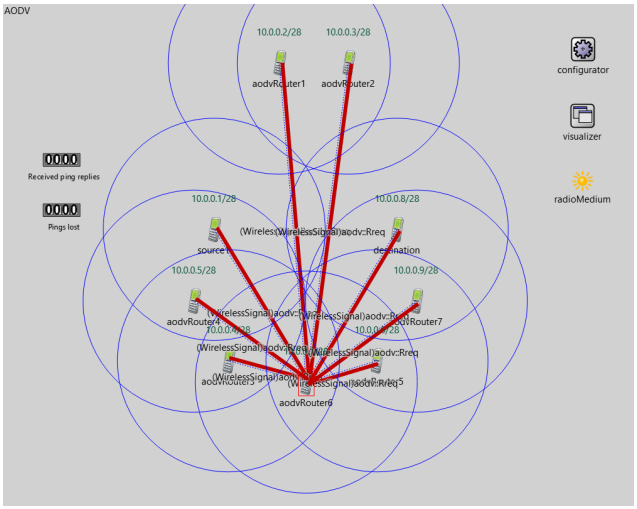
1. Explain, with a screenshot, how the path is created and how the first ping packet is sent.

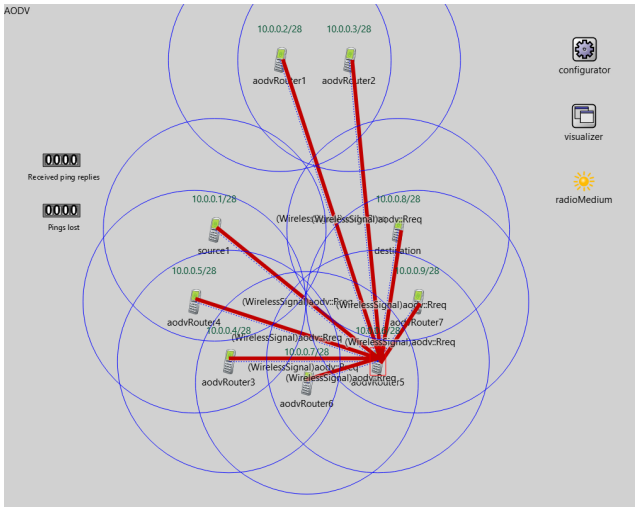
   The screenshots provided in this page and the following page demonstrate the creation of path and also shows how the first ping packet is sent.

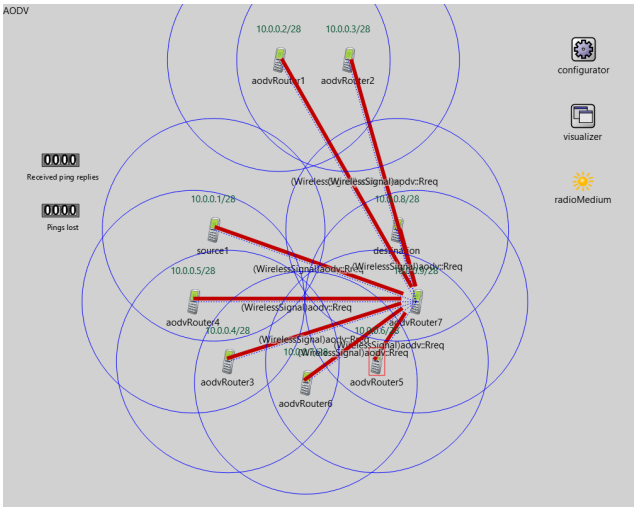












When a node wants to send a packet, and it doesn't know the route to the destination, it initiates route discovery, by sending an RREQ multicast message. The neighboring nodes record where the message came from and forward it to their neighbors until the message gets to the destination node. The destination node replies with an RREP, which gets back to the source on the reverse path along which the RREQ came. Forward routes are set up in the intermediate nodes as the RREP travels back to the source. An intermediate node can also send an RREP in reply to a received RREQ if it knows the route to the destination, thus nodes can join an existing route. When the RREP arrives at the source, and the route is created, communication can begin between the source and the destination.

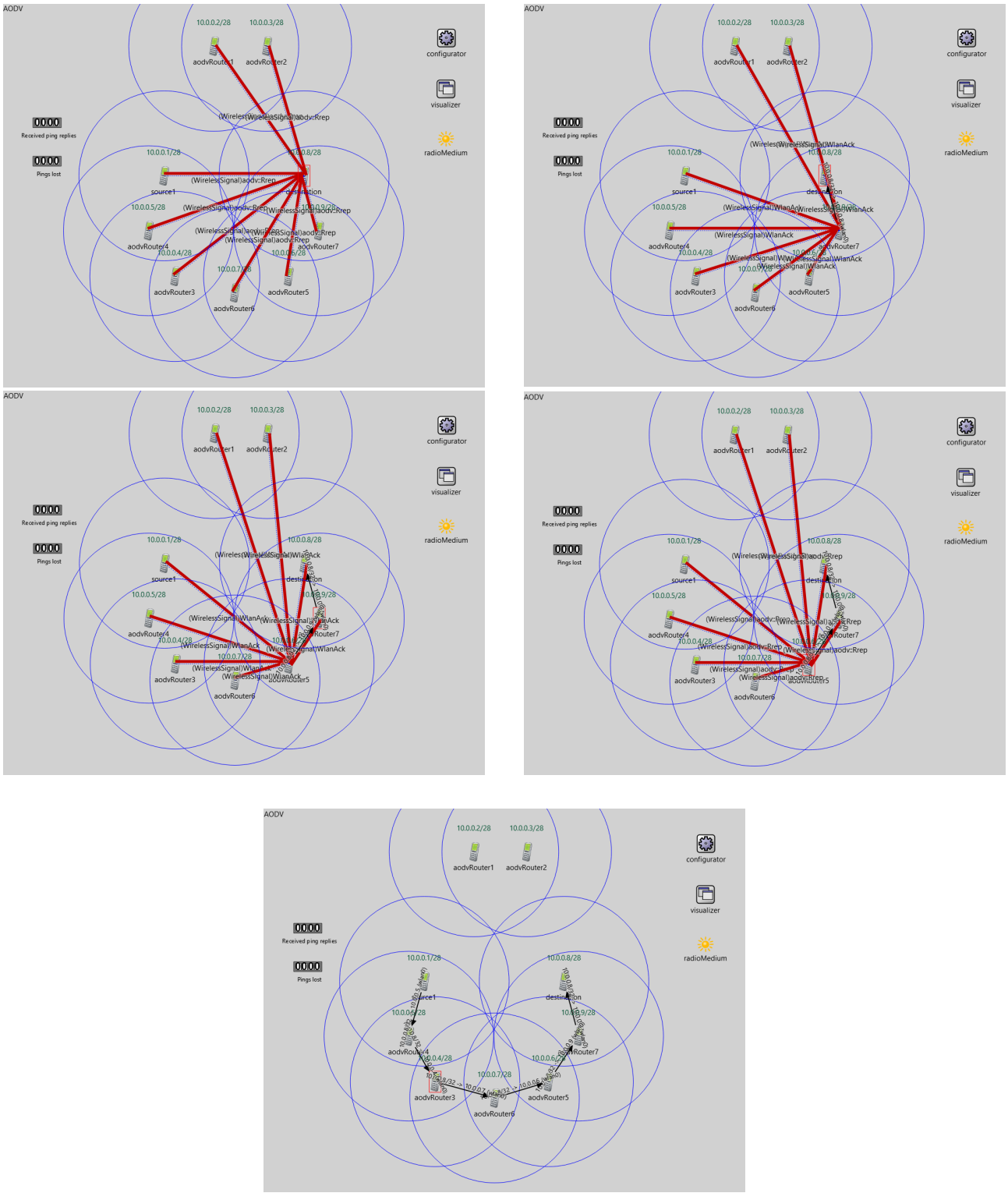


Figure 37: The process of path determination

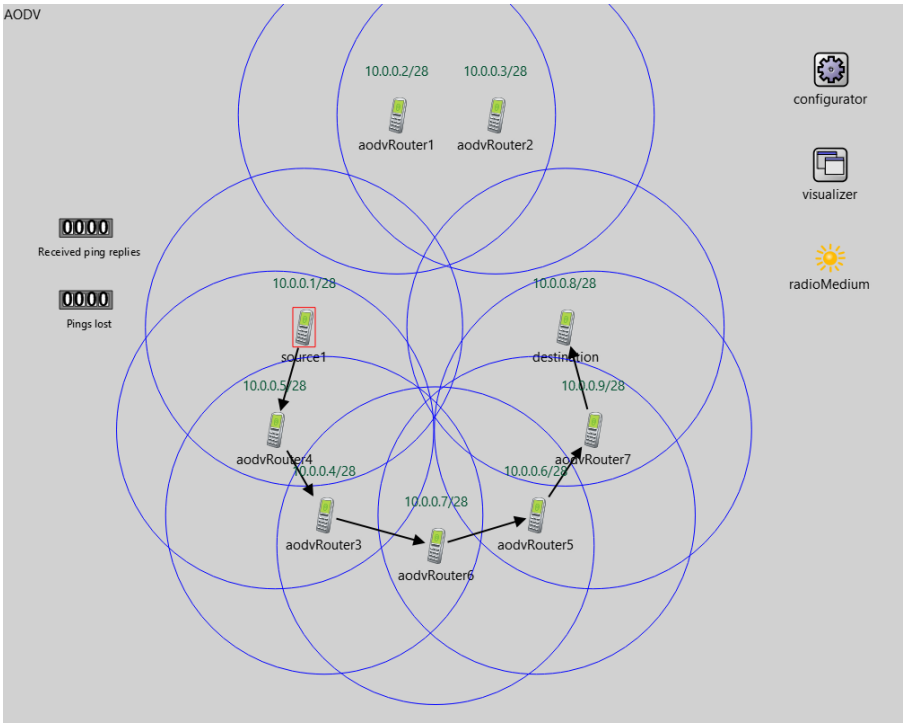


Figure 38: The structure of the path

2. explore the AODV router parameters in INET by answering the following *:

   (a) Why do we need jitter for broadcasts, and how does it help prevent collisions?

   In AODV, broadcasts are critical for operations like route discovery, where Route Request (RREQ) messages are sent to all nodes within range. However, in wireless ad-hoc networks, simultaneous transmissions from multiple nodes can lead to collisions, where signals interfere and messages are lost. To mitigate this, AODV incorporates a jitter mechanism, which introduces a small random delay before transmitting broadcast messages.

   The INET documentation for the AODV router specifies the following parameters related to jitter:

   maxJitter: Set to 5 milliseconds by default.

   jitter: Defined as uniform(0ms, maxJitter).

   (b) Why is the initial Time-To-Live (TTL) value set low, and why is it gradually incremented?

   The INET implementation provides specific parameters for TTL management:

   ttlStart: Set to 2, indicating the initial TTL value for a route request.

   ttlIncrement: Set to 2, the value by which TTL is increased for each retransmission.

   ttlThreshold: Set to 7, the maximum TTL before the network diameter is used for broadcasting.

   localAddTTL: Set to 2, used for calculating initial TTL in local repairs.

   The initial TTL being set low (2 hops) suggests a strategy to limit the initial scope of route discovery to immediate neighbors or nearby nodes. This minimizes network overhead by avoiding unnecessary flooding of the entire network, which is particularly beneficial in large or dense MANETs where excessive broadcasts could lead to congestion and increased latency.

*answers to this question were obtained from INET documentation and RFC 3561

If no route is found within this initial scope, the TTL is gradually incremented by 2 for each retransmission, expanding the search area incrementally. This process continues until the TTL reaches the threshold (7), after which the network diameter (a parameter set to 35 in INET, representing the maximum possible hops) is used. This approach ensures efficiency by first checking for routes to nearby destinations, which are more likely to be relevant in many scenarios, before expanding the search. It also aligns with AODV's on-demand nature, where routes are discovered only when needed, reducing unnecessary traffic.

(c) Under what conditions is a route deleted from the routing table, and what parameters influence this process?

The conditions for route deletion include:

Route Expiration: Each route has a lifetime, and if it is not refreshed or used within this period, it is considered expired and deleted. This prevents stale routes from being used, which is crucial in dynamic MANETs where topology changes frequently.

Link Break Detection: If a node detects a link break along an active route (e.g., due to node movement or failure), it generates a Route Error (RERR) message. This message is sent to the source node and any other nodes using the route, prompting them to invalidate and eventually delete the route from their tables.

Route Not Needed: While less explicitly stated, routes may also be deleted if they are no longer needed, such as when there are no pending packets to send to that destination, to free up space in the routing table.

The parameters influencing this process include:

activeRouteTimeout: Set to 3 seconds by default, this is the time after which an active route is considered expired if not used or refreshed. It ensures routes are periodically validated, especially in the presence of Hello messages for neighbor detection.

deletePeriod: Set to 5 * max(activeRouteTimeout, helloInterval), this determines how long an expired route is kept before being deleted. For example, if activeRouteTimeout is 3 seconds and helloInterval is 1 second, deletePeriod would be 15 seconds, allowing time for potential re-establishment.

helloInterval: Set to 1 second by default, this parameter, along with allowedHelloLoss (set to 2), influences route maintenance. The activeRouteTimeout must be greater than allowedHelloLoss * helloInterval to prevent premature deletion.

3. Analyze the key packet types used in the AODV protocol during the simulation:

(a) **Route Request (RREQ):** Examine the fields in RREQ packets, including: rreqId, hopCount, destSeqNum, ttl.

rreqId (Route Request Identifier): It is a unique identifier for the RREQ, generated by the source node and used along with the source node's IP address to uniquely identify a specific route discovery attempt. It prevents processing of duplicate RREQs, reducing network overhead and avoiding routing loops. When a node receives an RREQ, it checks the rreqId and source IP against its history to discard duplicates, ensuring only the first instance is processed or forwarded.

hopCount: The purpose of this field is to track the number of hops the RREQ has traversed from the source to the current node. It helps nodes evaluate the distance to the source and build reverse routes. Each node that forwards the RREQ increments this field.It is used to select the shortest path during route discovery and to update the routing table with the freshest route (based on hop count and sequence numbers).

destSeqNum (Destination Sequence Number): This field represents the source node's knowledge of the destination's sequence number, if available, or the last known sequence number. It ensures route freshness and loop prevention. Nodes compare the *destSeqNum* in the RREQ with their own routing table's destination sequence number (if they have a route to the destination). A higher or equal sequence number indicates a fresher route, prompting the node to generate an RREP or forward the RREQ.

ttl (Time-To-Live): It limits the number of hops an RREQ can travel before being discarded. It is responsible for controlling the scope of route discovery to prevent excessive flooding. Here, the *ttlStart* parameter (default 2) sets the initial TTL, which is incremented by *ttlIncrement* (default 2) up to *ttlThreshold* (default 7), and eventually to the network diameter (default 35). A node decrements the TTL before forwarding; if TTL reaches 0, the packet is dropped.

(b) **Route Reply (RREP):** Review the fields in RREP packets.

The key fields in RREP packets[†]:

Field	Description
hopCount	Tracks the number of hops from the destination to the source. This helps the source choose the best (shortest) route.
destAddress	The IP address of the destination.
destSeqNum	The destination's sequence number, indicating the freshness of the route. Helps the source determine whether to use this route.
lifetime	The time in milliseconds for which the route is considered valid. It's copied into the routing table on the source side.

Table 1: Fields in the AODV Route Reply (RREP) Packet

(c) What purpose do the Source and Destination Sequence Numbers serve in each routing packet?

Key Purposes of Sequence Numbers:

Loop Prevention: Sequence numbers make sure that newer routing information take the place of the older ones, preventing routing loops. If a node receives a packet with a lower sequence number than its current entry, it discards it.

Route Freshness: By comparing sequence numbers, nodes select the most recent route, critical in MANETs where topology changes frequently due to mobility of the nodes.

Route Selection: During route discovery, nodes use sequence numbers to prioritize fresher routes over older ones, even if the hop count is higher, ensuring robust routing decisions.

4. **activeRouteTimeout**

(a) Does AODV always send packets through the shortest path?

No.
AODV prefers the shortest (least-hop) path among the available and valid routes. However when multiple RREQs reach a destination, it responds to only the first valid one, even if a later one would result in a shorter route. Intermediate nodes forward only the first RREQ they see, discarding duplicates (to avoid flooding loops). The route established is based on the first received RREQ, which is often—but not guaranteed to be the shortest.
Therefore, AODV can lock into a suboptimal path if it receives an RREQ over a longer route before the optimal one arrives.

[†]obtained from INET documentaion

(b) What is the impact of the *activeRouteTimeout* parameter on the network's ability to select a better path?

The *activeRouteTimeout* parameter determines how long a route remains valid. A shorter timeout (e.g., 1 second) forces more frequent route discoveries, increasing the likelihood of finding a better path but adding control overhead. A longer timeout (e.g., 10 seconds) reduces overhead by keeping routes longer, but it may delay finding better paths if the network topology changes.

(c) What are the effects of using low and high values for this parameter?

As explained in the previous question, a longer *activeRouteTimeout* keeps routes in the routing table for a longer period, increasing the likelihood that an existing (potentially suboptimal) route is used instead of triggering a new route discovery. This can delay the discovery of a better (e.g., shorter or fresher) path, especially in dynamic networks where topology changes frequently.
On the other hand, shorter *activeRouteTimeout* forces routes to expire more quickly, prompting more frequent route discoveries via RREQs. This increases the chance of finding a better path (e.g., shorter or more stable) but at the cost of higher control overhead due to repeated broadcasts.

(d) Simulate the network using different *activeRouteTimeout* values and explain the impact.

activeRouteTimeout = 1 sec:

The overall number of RREP and RREQ packets generated increases significantly. Also the path changes when routers 1 and 2 come closer to the source and destination and eventually the path through those routers is selected. It is observed that some ping replies come back to the source with a great delay or even do not come back and the ping packet gets lost. All in all we can say that if the *activeRouteTimeout* is less than a normal value (2 or 3 seconds) instability might happen leading to packet loss or undesirable delays.

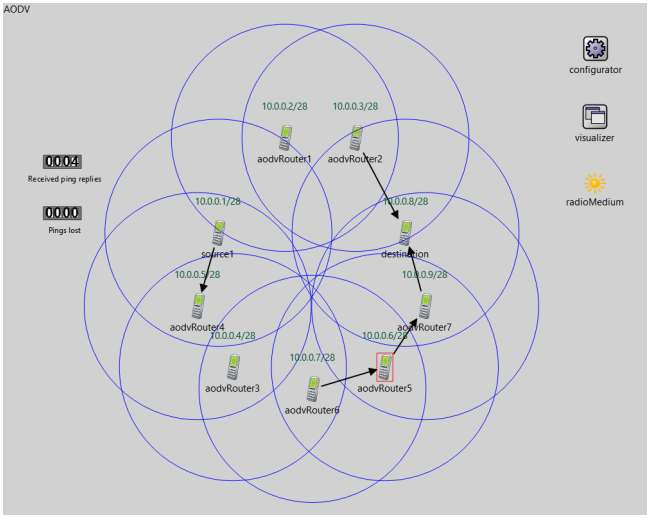


Figure 39: The selected path changes due to the short timeout

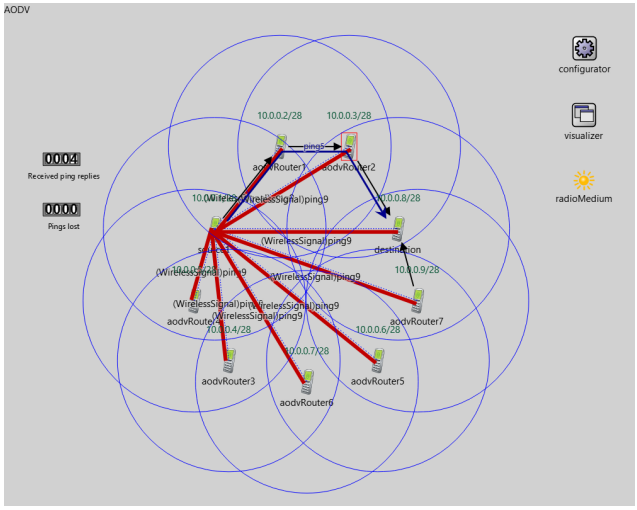


Figure 40: Some ping replies come back with a long delay

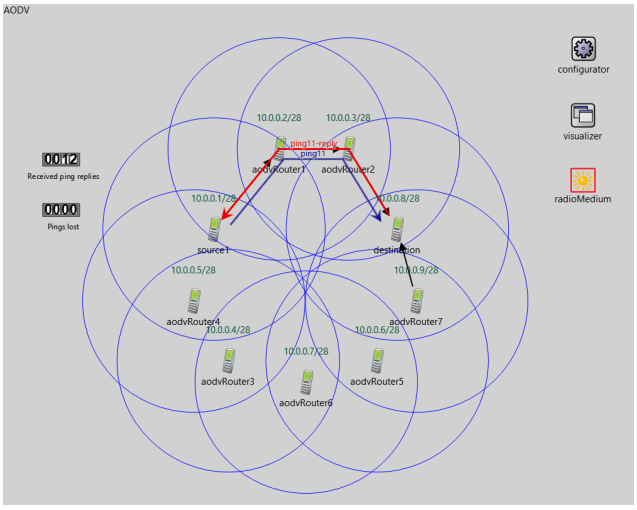


Figure 41: Some ping replies come back with a long delay

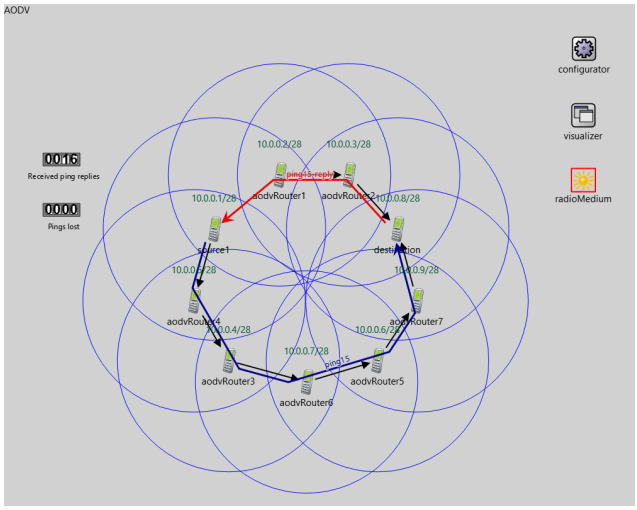


Figure 42: The ping and ping reply paths are not necessarily the same in this case

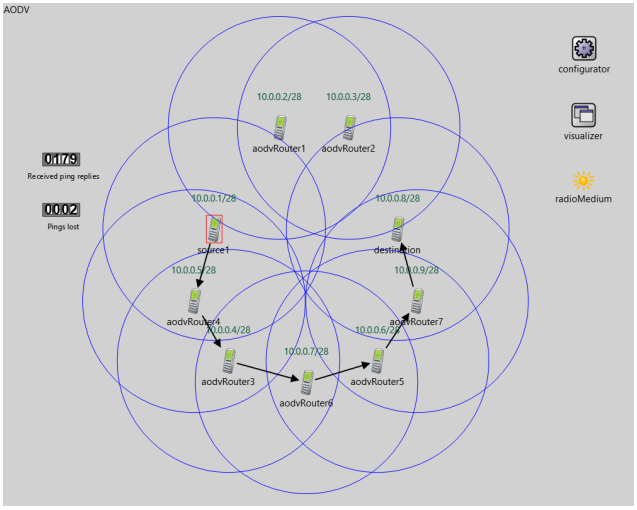


Figure 43: In this case we see some lost packets due to the instability

$activeRouteTimeout = 10$ sec:

In this case, because the timeout is large the routers in the path remain valid for a much longer time and thus we see much less RREQ and RREP packets. The selected route is not always the best path and when the two upper routers come closer to the source and destination there exists a better path.

5. **Route Error (RERR)**

   - Add mobility to *Router6* in order to break the established route.
   - Observe the scenarios where RERR packets are generated.

   As we can see in the screenshots below, after adding a 20 mps linear mobility with uniform(0deg, 360deg) movement heading to router6, the router starts moving until router5 stays out of its communication range. After this router6 broadcasts the ping packet several times and then eventually sends a RERR back to router3. Then the routers send the RERR packet back to the source. Following this, the routers try to find a path to send the ping packet via sending several RREQ and RREP packets until when the upper router enter the communication range of the source and destination. Afterwards the source uses the path through the upper routers and sends the ping packets.

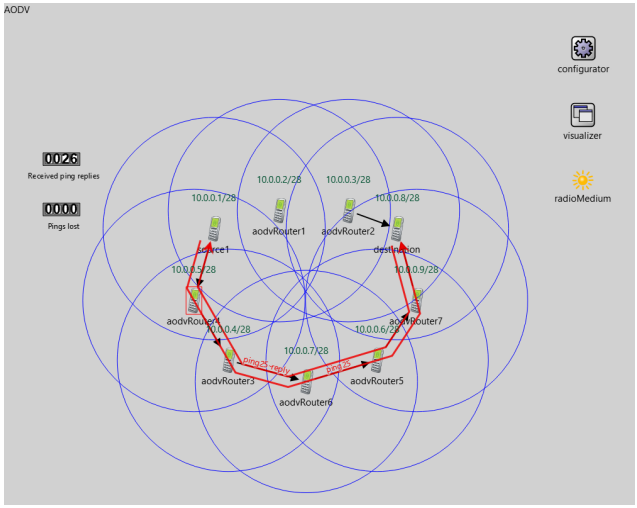


Figure 44: In this case the path and the router remain valid for a long time

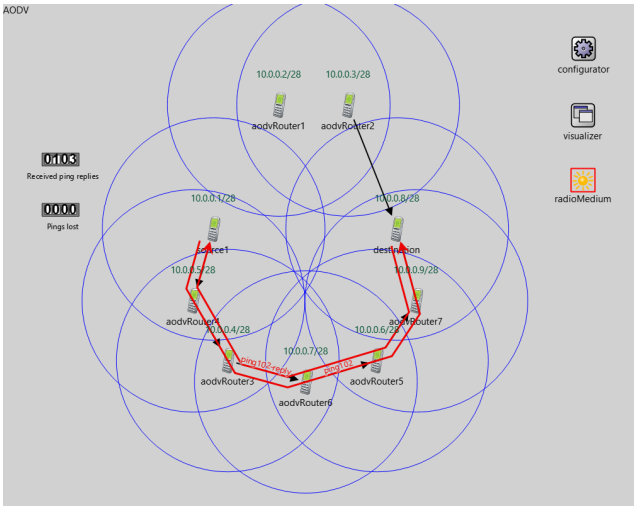


Figure 45: After a long time there is no lost packet and the network is stable

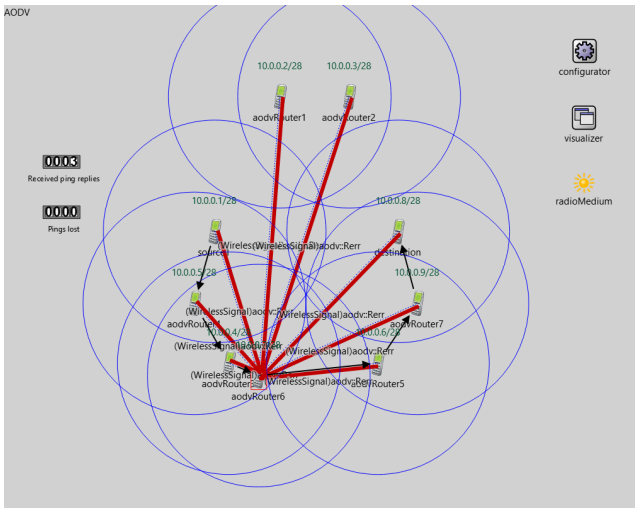


Figure 46: Router6 sends a RERR packet to router3 because router5 is out of its range

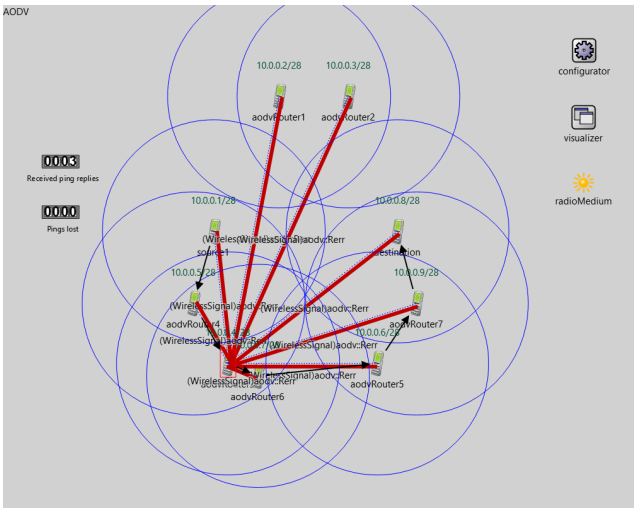


Figure 47: Router3 passes the RERR packet to router4 so that it would be forwarded to the source

6. **Hello Packets**

   (a) What is the purpose of Hello packets, and why are they disabled by default in some configurations?

   Hello packets are small, periodic unicast or local broadcast messages that allow AODV nodes to detect the presence or absence of neighboring nodes.
   Hello packets let nodes confirm their 1-hop neighbors are still within range, even if no data or control traffic is being exchanged. If a node stops receiving Hello packets from a neighbor for a certain time (*allowedHelloLoss* times *helloInterval*), it considers the link broken and triggers RERR. Routes that rely on neighbor connectivity can be refreshed or invalidated based on Hello reception.

   In default AODV implementations, the protocol works with link layer detection rather than Hello packets, suggesting that alternative mechanisms can handle neighbor discovery. For instance, in networks using IEEE 802.11, link layer notifications (e.g., absence of ACKs or failure to get CTS after RTS) can detect link breakages, reducing the need for Hello packets. Besides that, disabling Hello packets can reduce network overhead, saving bandwidth and processing power, which is particularly beneficial in resource-constrained environments like MANETs.

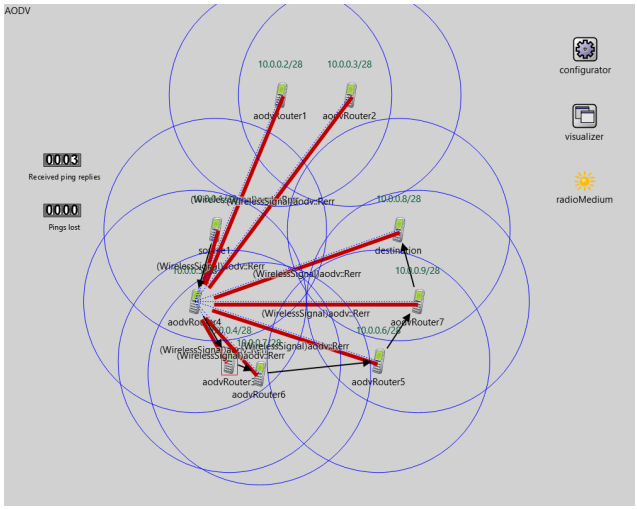


Figure 48: Router4 then gives the RERR packet to the source node

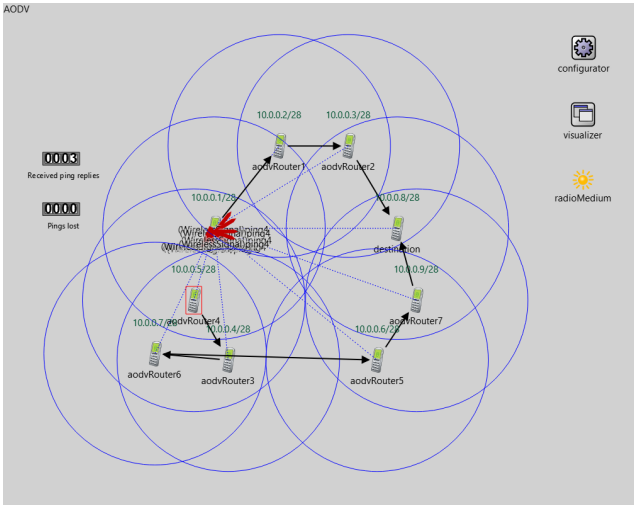


Figure 49: The nodes try to find a route until the upper routers get close enough

(b) Modify the ping app to observe the effect of hello packets. Configure the ping app to ping 3 times initially. After 3 pings, do not ping again until the destination moves back. Once the destination has moved back, ping again.

(c) Are Hello packets forwarded like RREQ messages?

No. Hello packets are not forwarded like RREQ messages. Hello messages are broadcast with TTL=1 which means they are only received by immediate neighbors and are not propagated further through the network.